



Introducción a CSS

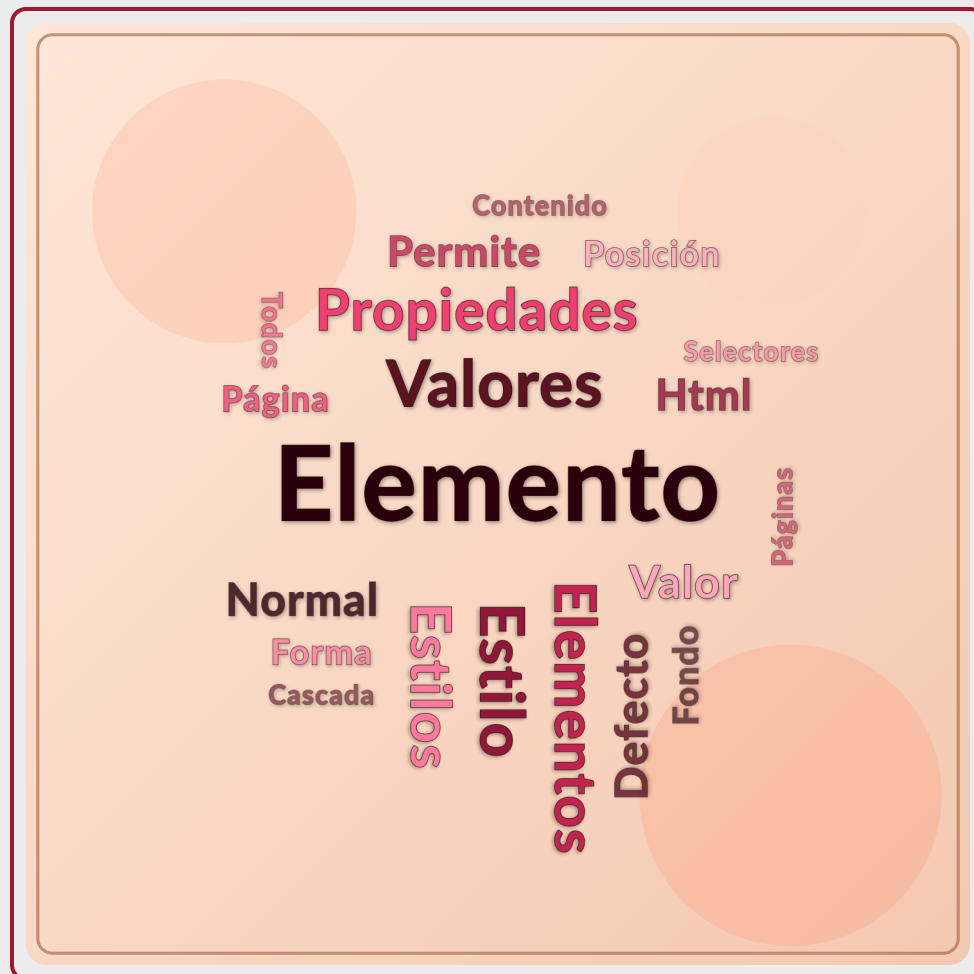
Universidad de Sevilla

Departamento de Lenguajes y Sistemas
Informáticos

Carlos Arévalo, Daniel Ayala, José Calderón,
Margarita Cruz, Inma Hernández, David Ruiz

UNIVERSIDAD DE SEVILLA

Objetivos



Contenidos

Introducción

Hojas de estilo en cascada (CSS)

Sintaxis CSS

Maquetación CSS



Antes / Después

1993

Advertise on Aliweb

Aliweb Home | September 11th, 2001

Aliweb

Search

Reset

ADVANCED OPTIONS: Case Sensitive: ☐
Display Fields Other Than Title? ☒ Description ☐ Keyword ☐ URL ☐ Other
Select Search Fields ☒ Title ☒ Description ☒ Keyword ☒ URL
Limit Results To: (20 matches) Restrict to Domain: (none)
Search Type: (Whole Words) Record Type: (Any)

AliLinks - Links to the web's best sites all on one easy to navigate page!

The Web6000 Network makes it easy and affordable. [click here.](#)

Computing

Jump to Section TOP

Computing: Broadband, Linux, Music, Streaming, Recreation, Research, Shopping, SOFTWARE

Browsers Etc.: Amaya | Amaya2000 | Crome | Microsoft IE | Firefox & Thunderbird | FlashPlayer | Maxthon | Netscape (old) | Opera | PDF Reader | Pegasus Mail | QuickTime | RealPlayer | Safari | Shockwave

CGI/Java: CGI Resource | Extropia | Gamelan | Java Boutique | JavaScript | Matt's | Webscripts |

Companies: AMD | Apple | Compaq | Citrix | Dell | Digital | Electro Service | Gateway | HP | Intel | Winchip

Free Services: Announce | Attid | ATC | Ezeemote | Shotgun | Banners | Link Exchange | Smartclicks | Chat | ActiveWorlds | Chatlist | Globe | Talk City | WBS | Yack! | Email | Hotmail | Juno | MailExcite | Hosting | AngelFire | Fortune City | Geocities | Trillpark | Tspod | Web6000 | Xoom

Games: A.Vault | Gamecenter | Game Finder | GamePen | GamePower | G.Domain | Games.net | GameSpot | Happy Puppy | OGS

Site Help: Absolute | Builder.com | DJ Quad | developer.com | Dev_Zone | HOME | HTMLgoodies | HTML Guide | WebDeveloper | reallybig.com | Webmonkey | Webreference | Web Review

Site Tools: Color Chart | Cool Tool | Dr. HTML | FreeForm | GIF Wizard | NetMechanic | SBN Gallery | Site Garage

Software: Dave Central | Download | Extreme Mac | Filemine | FileFile | Filaz | Freewareplus | Freewareweb | Jumbo | Nonags | Softseek | Stroud's | Tucows | Windrivers | Winfiles | WinSite | ZDnet | Hotfiles

Support: Newbie-U | Microsoft | PC-Help | Support Help | Whatis

Entertainment

Jump to Section TOP

Entertainment: Linux, Music, Streaming, Recreation, Research, Shopping, SOFTWARE

Movies: BoxOffice | Cinemachine | Film.com | Film Scouts | In Theaters | IMDb | MovieFinder | MovieLink | Mr. Cranky | Reel

Movie Studios: Buena Vista | Disney | Dimension | Fox | Fox Searchlight | Hollywood | MGM/UA | Miramax | New Line | October | Paramount | Sony | Universal | Warner Bros

Music Ezines: Allstar | Classical | CMJ Online | Ear.1 | MTV | RollingStone | SonicNet | VibeOnline

Music Concerts: LiveConcerts | Live Online | Pollstar | Ticketmaster | WILMA

Music News: Billboard | HitsWorld | Imusic | Newswire

Music Search: IUMA | JazzCentral | LyricServer | MusicCentral | MusicSearch | UBL | UnfURLed | WallOfSound

Stars: Cybersleaze | eQlive | E! Online | Ent. Asylum | E-Weekky | Enquirer | Hollywood | Mr. Showbiz | People | Variety

TV: click TV | GIST | Phocus | Rock on TV | Total TV | TV Guide | UltimateTV

TV Networks: ABC | CBS | Disney | FOX | ESPN | HBO | NBC | PBS | WB | UPN | SciFi

Living

Jump to Section TOP

Computing: Entertainment, Linux, Music, Streaming, Recreation, Research, Shopping, SOFTWARE

Art: A.Resource | CultureFinder | Fine Site | The Guide | Playbill | Buy/Sell | DoubleTake

Education: edu | A.S.D. | AskERIC | CollegeNet | Paternors | University | Web 99

Family: Family.com | Parenthood | Parent Soup | Parents.com | ParentTime | WholeFamily

Food: BettyCrocker | Digital Chef | DineNet | Epicurious | Food TV | I. Kitchen | Kitchen Link | Meals 4U | Phyl's | Star Chefs | Zagat

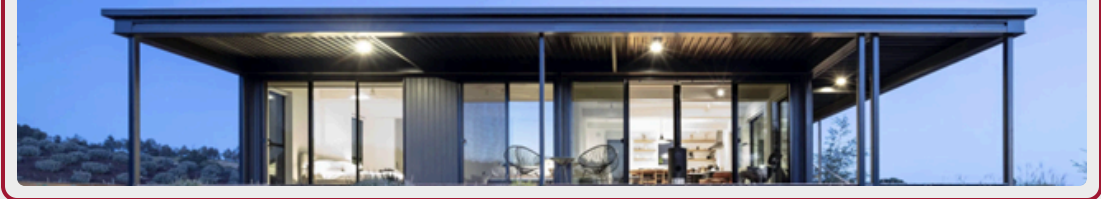
2026

CONNECT HOMES

HomeownersFor DevelopersSupportive HousingAll Products

AboutGalleryContact UsCareers

BRILLIANTLY SIMPLE, BEAUTIFULLY MODULAR PREFAB HOMES.



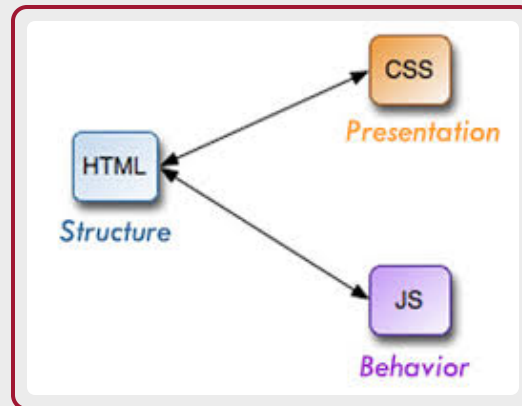
Cascading Style Sheets

Estándar W3C: <https://www.w3.org/Style/CSS/Overview.en.html>

Objetivo: describir la **presentación** de un documento escrito en un lenguaje de marcado, como HTML.

Permite la separación de contenido (información que se muestra y su estructura) y presentación (cómo se muestra la información: layout, colores, fuentes, tamaños...).

Cascading (en cascada): se refiere al orden de prioridad en que se aplican las reglas de estilo cuando hay más de una regla asociada a un elemento HTML en concreto.



Ventajas de la separación presentación-contenido

Simplifica el HTML de las páginas web.

Permite dar una apariencia homogénea a un sitio web al aplicar los mismos estilos a todas sus páginas.

Permite compartir un mismo estilo entre páginas de distintos sitios, usando el mismo fichero de estilos (.css)

Amplía las posibilidades de presentación de HTML al permitir mucho más control.

Permite presentar el mismo contenido de diferentes formas sin tener que modificarlo.
Ejemplos:

- Mostrar la misma página con distinta presentación según el dispositivo en que se está mostrando (PC, móvil, tablet...)
- Tener distintas versiones de una misma página con distintos estilos de presentación

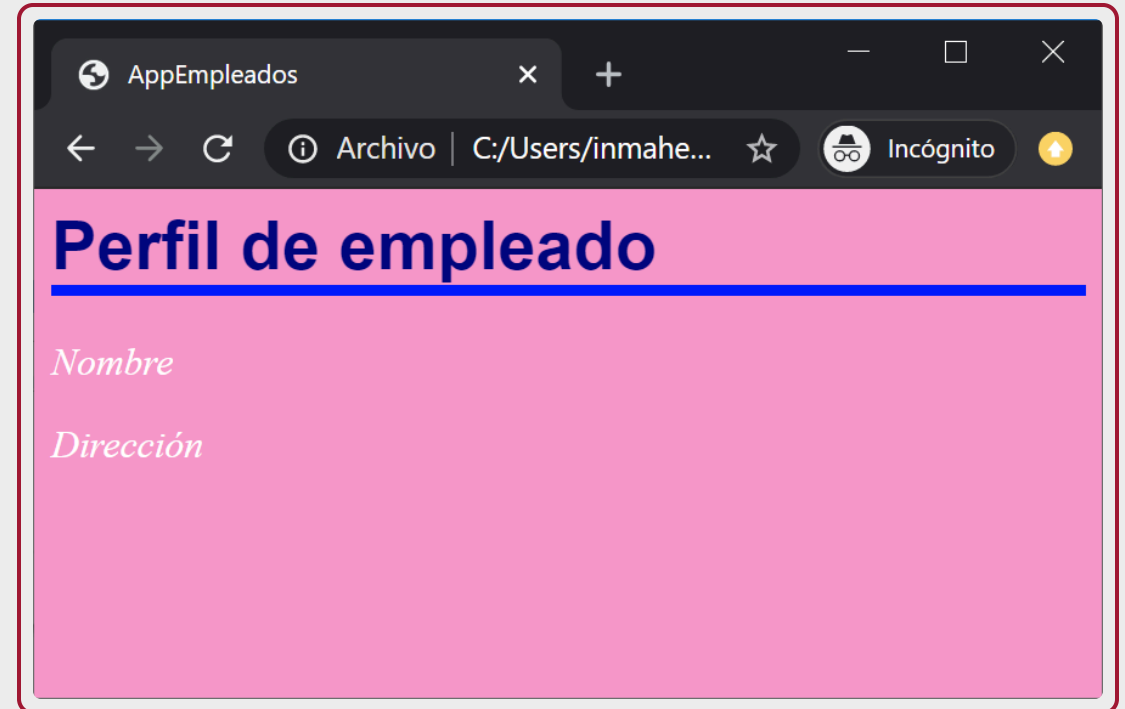
Ejemplo de estilos CSS

```
<html>
  <head>...</head>
  <body>
    <h1>Perfil de empleado</h1>
    <p>Nombre</p>
    <p>Dirección</p>
  </body>
</html>
```

```
body {
  background-color: #ff99cc;
}

h1 {
  color: navy;
  font-family: sans-serif;
  border-bottom-width: 5;
  border-bottom-style: solid;
  border-bottom-color: blue;
}

p {
  color: white;
  font-style: italic;
  font-size: large;
}
```



No todos los navegadores muestran los estilos de la misma forma

Timeline del estándar CSS

La primera versión del estándar es de diciembre de 1996, conocido como CSS nivel 1.

Posteriormente, en mayo de 1998 se aprueba definitivamente la especificación CSS nivel 2.

Desde junio de 2005 está disponible el borrador de CSS 2.1. En Junio de 2011 se aprueba el definitivo.

Desde finales de 1999 se trabaja en CSS3 que se descompone en módulos (>50 en desarrollo). En 2013 se aceptan 5 de ellos como recomendación (standard).

Los módulos han ido evolucionando de forma independiente, algunos llegando a nivel 4 (se basan en elementos de nivel 3).

Snapshots: colecciones de módulos estables. Último snapshot: 2025 (<https://www.w3.org/TR/css-2025/>)

Completed work	Status	Upcoming	
CSS Snapshot 2023	NOTE		00
CSS Snapshot 2022	NOTE		00
C	NOTE		00
C	NOTE		00
C	NOTE		00
C	NOTE		00
C	NOTE		00
C	NOTE		00
CSS Snapshot 2007	NOTE		00
CSS Color Level 3	REC	REC	00
CSS Namespaces	REC	REC	00
Selectors Level 3	REC	REC	00
CSS Level 2 Revision 1	REC	REC	00
Media Queries	REC	REC	00
CSS Style Attributes	REC	REC	00
CSS Cascading and Inheritance Level 3	REC	REC	00
CSS Fonts Level 3	REC	REC	00
CSS Writing Modes Level 3	REC	REC	00
CSS Basic User Interface Level 3	REC	REC	00
CSS Box Model Level 3	REC	REC	00
CSS Containment Level 1	REC	REC	00
Stable drafts	Status	Upcoming	
CSS Backgrounds and Borders Level 3	CRD	CR	00
CSS Conditional Rules Level 3	CR	CR	00
CSS Multi-column Layout Level 1	CR	PR	00
CSS Values and Units Level 3	CR	PR	00
CSS Flexible Box Layout Level 1	CR	PR	00
CSS Counter Styles Level 3	CR	PR	00

*Nota sobre los status W3C

Working Draft (WD)

Documento que publica el W3C para que sea revisado por la comunidad (miembros del W3C, otras organizaciones de carácter técnico y el público en general).

Candidate Recommendation (CR)

Documento que el W3C considera ampliamente revisado y satisface los requisitos técnicos. Se publica para animar a los expertos a que implementen sobre el estándar y así obtener feedback

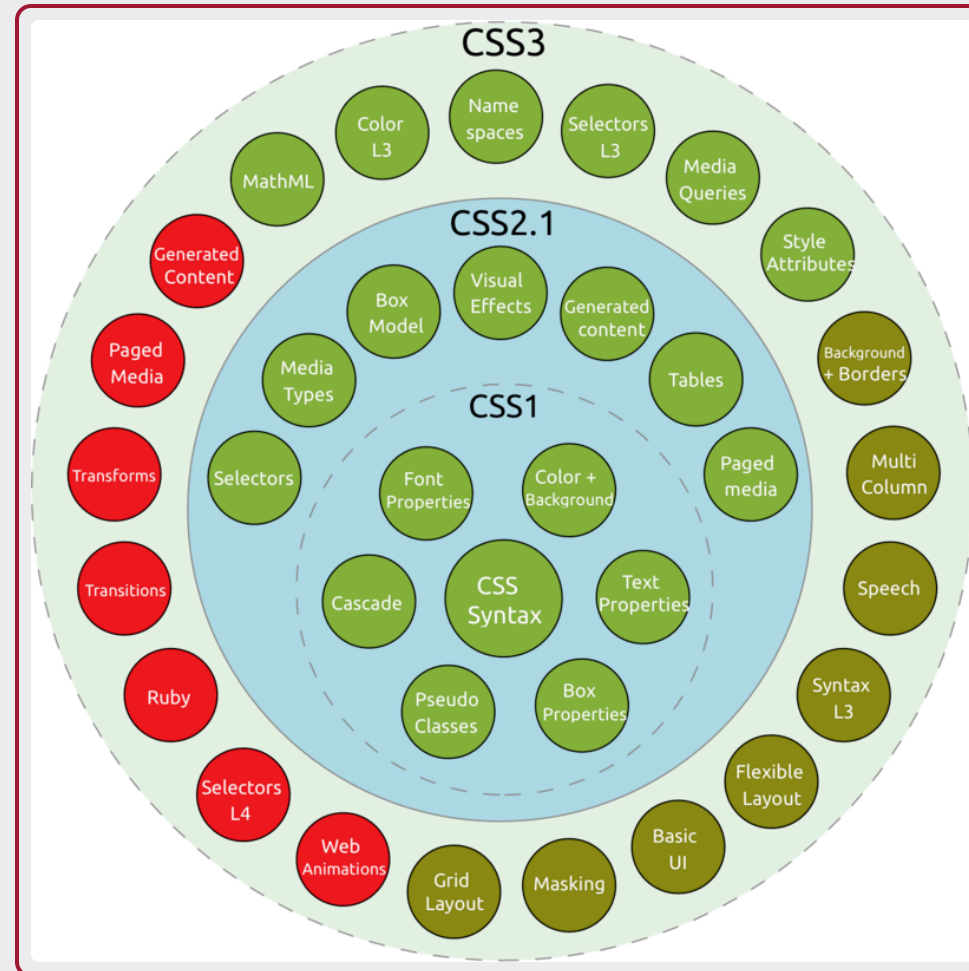
Proposed Recommendation (PR)

Informe técnico maduro, que ha sido revisado desde un punto de vista teórico y de implementación, y que se envía al W3C Advisory Committee para su aprobación final.

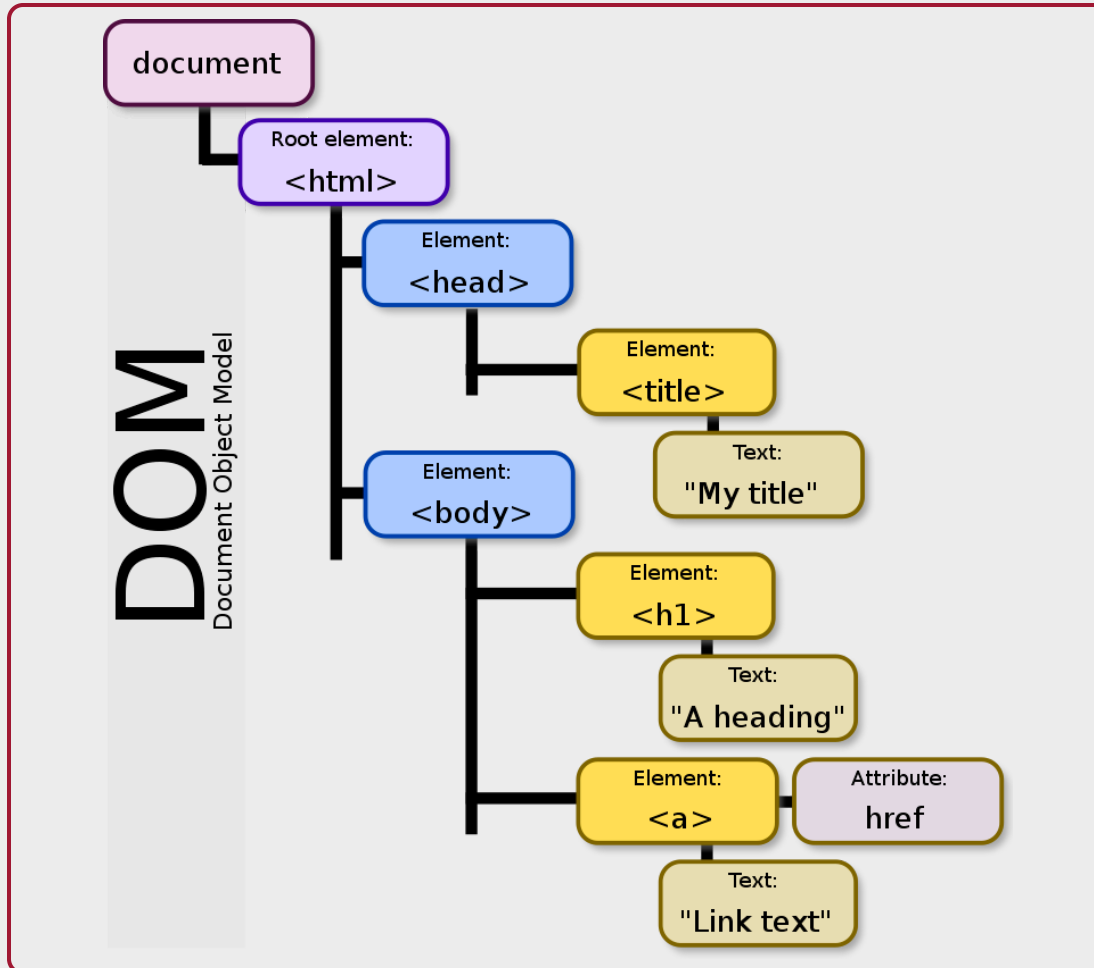
W3C Recommendation (REC)

Especificación o conjunto de normas que ha recibido la aprobación del W3C. Equivalente al concepto de "estándar" de otras organizaciones

Niveles CSS



Árbol DOM



DOM: Document Object Model

DOM transforma todos los documentos HTML en un conjunto de elementos llamados nodos, que están interconectados y que representan los contenidos de las páginas web y las relaciones entre ellos.

Permite acceder de forma programática a los nodos y su contenido

Contenidos

Introducción

Hojas de estilo en cascada (CSS)

Sintaxis CSS

Maquetación CSS



Formas de introducir CSS en HTML

Atributo style

Hoja de estilos interna

Hoja de estilos externa

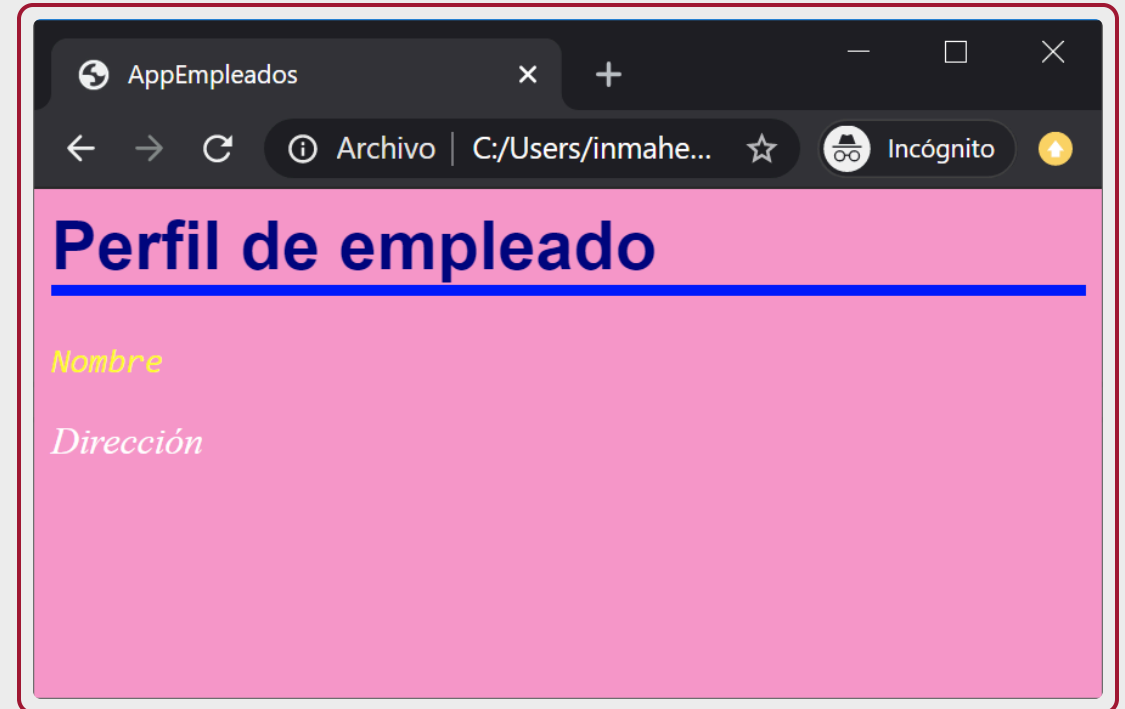


Atributo Style

Se puede especificar el estilo de un elemento HTML mediante el atributo `style`.

No es recomendable porque no permite su reutilización.

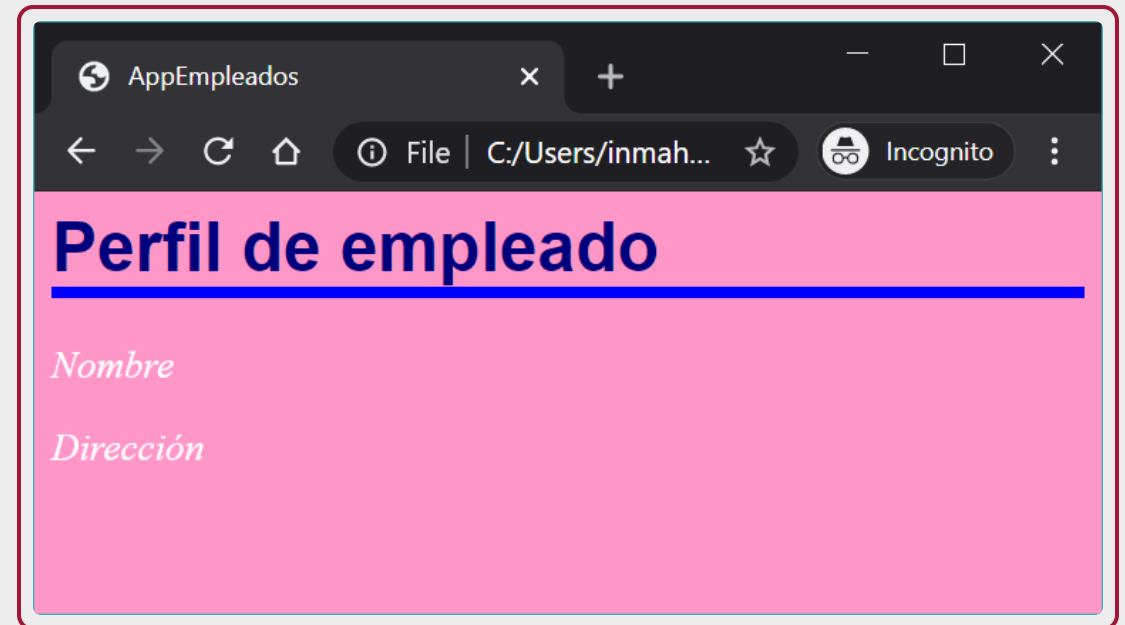
```
<html>
  <head>...</head>
  <body>
    <h1>Perfil de empleado</h1>
    <p style="font-family:monospace; color:yellow">
      Nombre
    </p>
    <p>Dirección</p>
  </body>
</html>
```



Esta forma tiene la prioridad más alta en la cascada

Hoja de estilos interna

```
<html>
  <head>
    <title>AppEmpleados</title>
    <style type="text/css">
      body { background-color: #ff99cc; }
      h1 {
        color: navy;
        font-family: sans-serif;
        border-bottom-width: 5;
        border-bottom-style: solid;
        border-bottom-color: blue;
      }
      p {
        color: white;
        font-style: italic;
        font-size: large;
      }
    </style>
  </head>
  <body>
    <h1>Perfil de empleado</h1>
    <p>Nombre</p>
    <p>Dirección</p>
  </body>
</html>
```



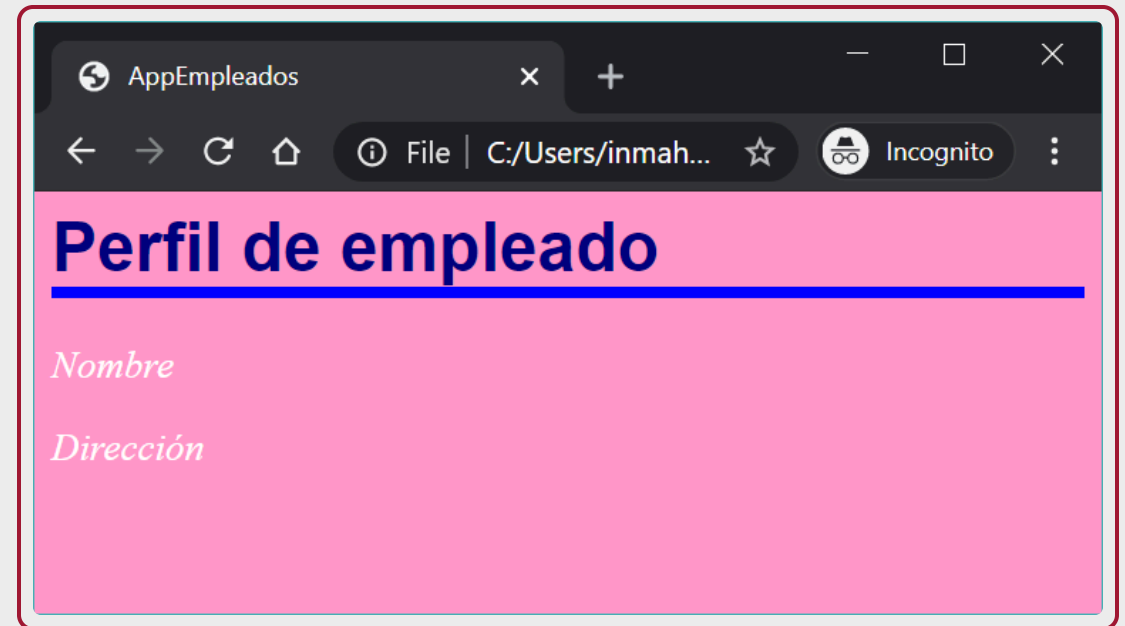
Se puede incluir una hoja de estilos en una cabecera HTML (head) mediante el elemento `<style>`.

Es mejor que usar el atributo `style` pero no permite reutilizar los estilos en otras páginas.

Hoja de estilos externa

```
<head>
  <link rel="stylesheet" type="text/css"
        href="empleado.css" />
</head>
<body>
  <h1>Perfil de empleado</h1>
  <p>Nombre</p>
  <p>Dirección</p>
</body>
```

```
/* empleado.css */
body {
  background-color: #ff99cc;
}
h1 {
  color: navy;
  font-family: sans-serif;
  border-bottom-width: 5;
  border-bottom-style: solid;
  border-bottom-color: blue;
}
p {
  color: white;
  font-style: italic;
  font-size: large;
}
```



Se usa el elemento `<link>` en la cabecera.
Es la forma recomendada, ya que permite reutilizar la misma hoja de estilo en varias páginas.

Contenidos

Introducción

Hojas de estilo en cascada (CSS)

Sintaxis CSS

Maquetación CSS



Reglas de estilo

Una hoja de estilo consiste en una lista de reglas (rules)

Una regla o conjunto de reglas consiste en uno o más selectores (selectors), y un bloque declarativo (declaration block)

A CSS ruleset (or rule):

```
div p, #id:first-line
```

```
{
```

```
background-color : red ;
```

```
background-style : none
```

```
}
```

Group of selectors

Declarations block

Selectores

Un selector declara a qué elemento(s) de la página web afecta la declaración del estilo, y puede ser de distintos tipos:

- Todos los elementos de un mismo tipo. Ejemplo: todos los elementos `<h2>`
- Todos los elementos que tienen un valor concreto de algún atributo, siendo el atributo:
 - `id` : identifica a un elemento de la página
 - `class` : identifica a uno o varios elementos de la página
 - Ambos son case-sensitive, empiezan con letras y pueden tener caracteres alfanuméricos, guiones y guiones bajos.
- Elementos que están en una determinada posición relativa con respecto a otros según el árbol DOM.

Tipos de selector



Declaraciones de estilo

La declaración está formada por una lista de pares `propiedad:valor` separados por punto y comas.

Algunas propiedades admiten valores compuestos (separados por espacios) o valores alternativos (separados por comas).

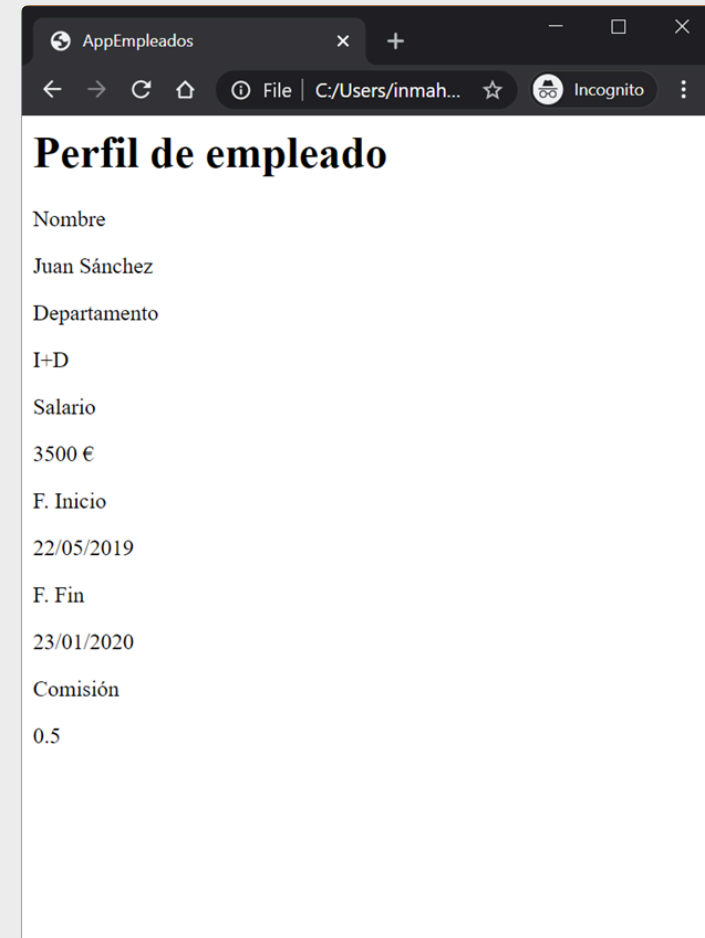
Se pueden importar (incluir) otras hojas de estilo.

Se pueden introducir comentarios estilo C tanto dentro como fuera de las declaraciones.

```
/* comentario */  
selector1, selector2, ..., selectorM {  
  propiedad1: valor1; /* comentario */  
  propiedad2: valor2;  
  ... ;  
  propiedadN: valorN  
}  
@import url(camino_hoja_externa.css);
```

Ejemplo: sin CSS

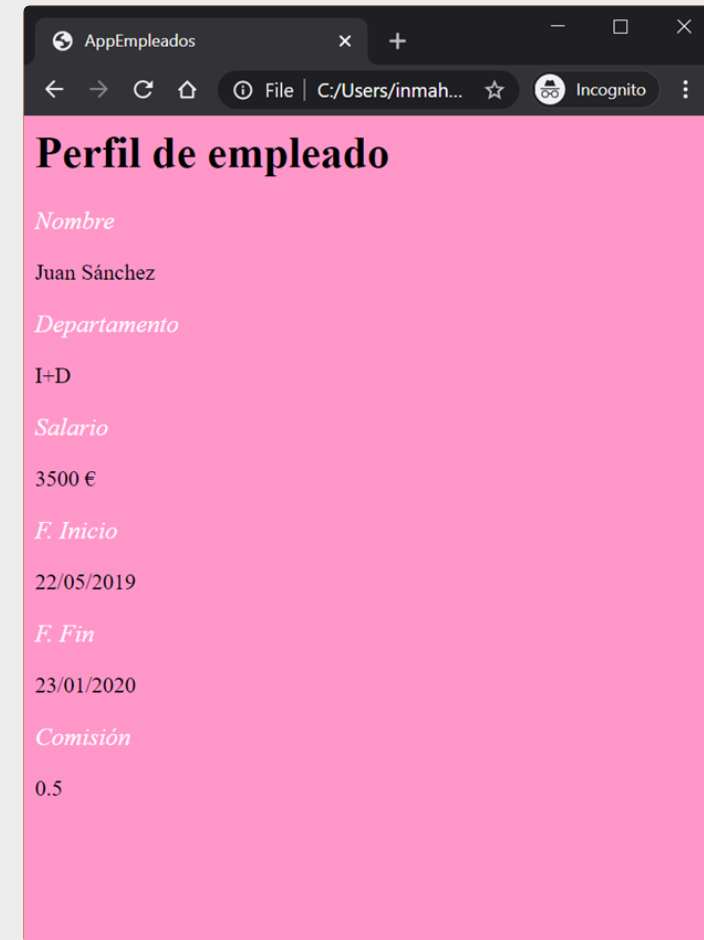
```
<html>
  <head>
    <title>AppEmpleados</title>
  </head>
  <body>
    <h1>Perfil de empleado</h1>
    <p>Nombre</p> Juan Sánchez
    <p>Departamento</p> I+D
    <p>Salario</p> 3500 €
    <p>F. Inicio</p> 22/05/2019
    <p>F. Fin</p> 23/01/2020
    <p>Comisión</p> 0.5
  </body>
</html>
```



Ejemplo: selectores `body` y `p`

```
<html>
  <head>
    <title>AppEmpleados</title>
    <link rel="stylesheet" type="text/css"
          href="empleado.css" />
  </head>
  <body>
    <h1>Perfil de empleado</h1>
    <p>Nombre</p> Juan Sánchez
    <p>Departamento</p> I+D
    <p>Salario</p> 3500 €
    <p>F. Inicio</p> 22/05/2019
    <p>F. Fin</p> 23/01/2020
    <p>Comisión</p> 0.5
  </body>
</html>
```

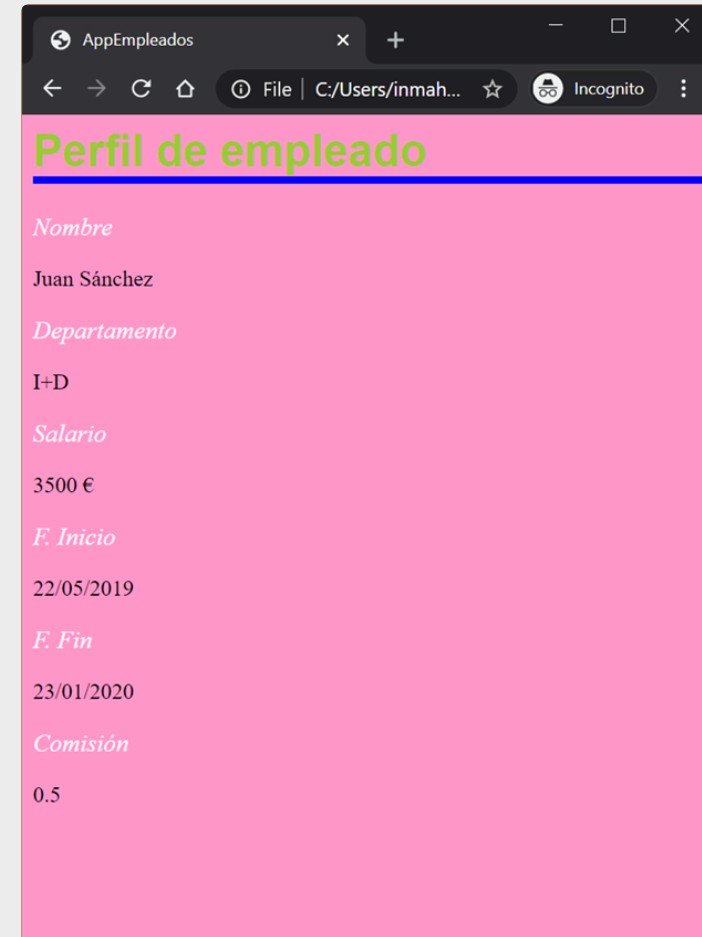
```
/* empleado.css */
body {
  background-color: #ff99cc;
}
p {
  color: white;
  font-style: italic;
  font-size: large;
}
```



Ejemplo: selector **h1**

```
<html>
  <head>
    <title>AppEmpleados</title>
    <link rel="stylesheet" type="text/css"
          href="empleado.css" />
  </head>
  <body>
    <h1>Perfil de empleado</h1>
    ...
  </body>
</html>
```

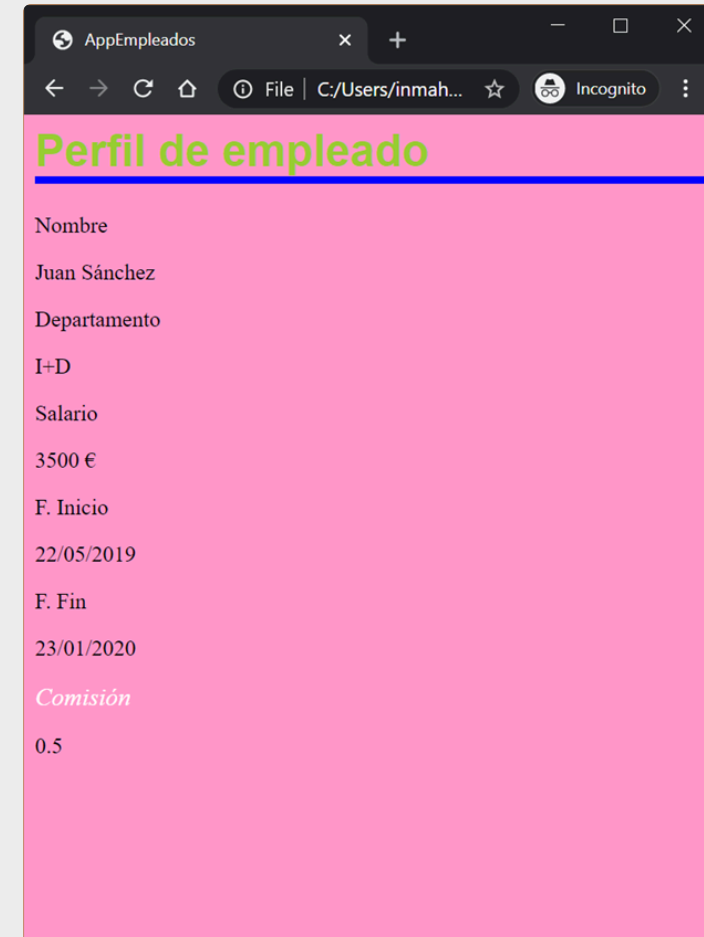
```
/* empleado.css */
...
h1 {
  color: yellowgreen;
  font-family: sans-serif;
  border-bottom-width: 5;
  border-bottom-style: solid;
  border-bottom-color: blue;
}
...
```



Ejemplo: posición relativa em dentro de p

```
<html>
  <head>
    <title>AppEmpleados</title>
    <link rel="stylesheet" type="text/css"
      href="empleado.css" />
  </head>
  <body>
    <h1>Perfil de empleado</h1>
    <p>Nombre</p> Juan Sánchez
    ...
    <p><em>Comisión</em></p> 0.5
  </body>
</html>
```

```
/* empleado.css */
...
p em {
  color: white;
  font-style: italic;
  font-size: large;
}
...
```

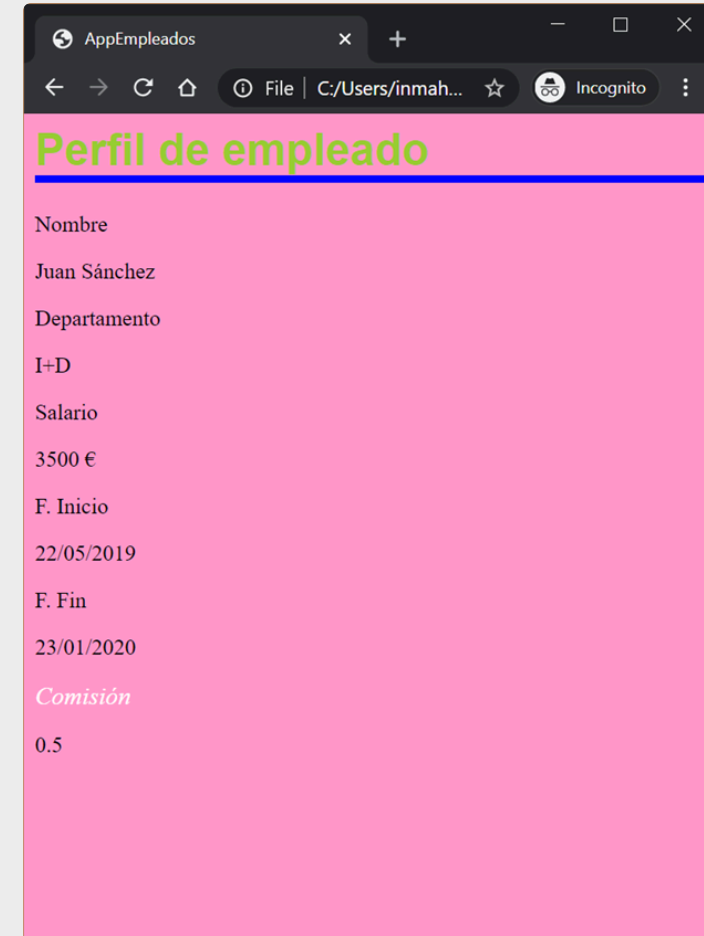


Ejemplo: class `.rotulo` (I)

```
<html>
  <head>
    <title>AppEmpleados</title>
    <link rel="stylesheet"
          type="text/css" href="empleado.css" />
  </head>
  <body>
    <h1>Perfil de empleado</h1>
    ...
    <p class="rotulo">Comisión</p> 0.5
  </body>
</html>
```

```
/* empleado.css */

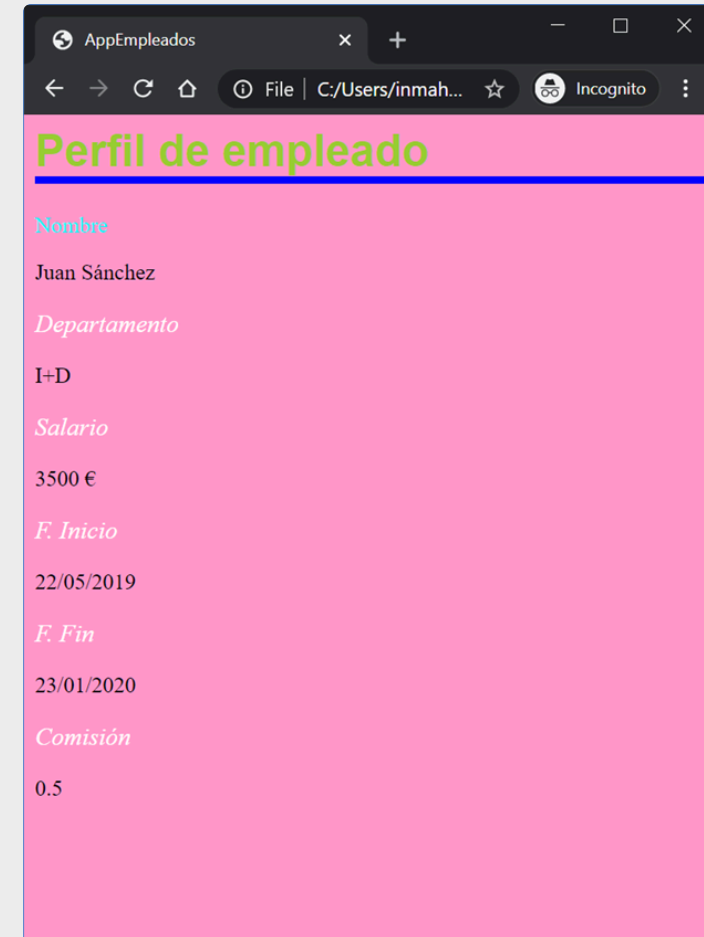
body {
  ...
}
h1 {
  ...
}
.rotulo {
  color: white;
  font-style: italic;
  font-size: large;
}
```



Ejemplo: class `.rotulo` (II)

```
<html>
  <head>
    <title>AppEmpleados</title>
    <link rel="stylesheet"
          type="text/css" href="empleado.css" />
  </head>
  <body>
    <h1>Perfil de empleado</h1>
    <p id="nombre">Nombre</p> Juan Sánchez
    <p id="nombre"> Departamento</p> I+D
    <p class="rotulo">Salario</p> 3500 €
    <p class="rotulo">F. Inicio</p> 22/05/2019
    <p class="rotulo">F. Fin</p> 23/01/2020
    <p class="rotulo">Comisión</p> 0.5
  </body>
</html>
```

```
/* empleado.css */
.rotulo {
  color: white;
  font-style: italic;
  font-size: large;
```

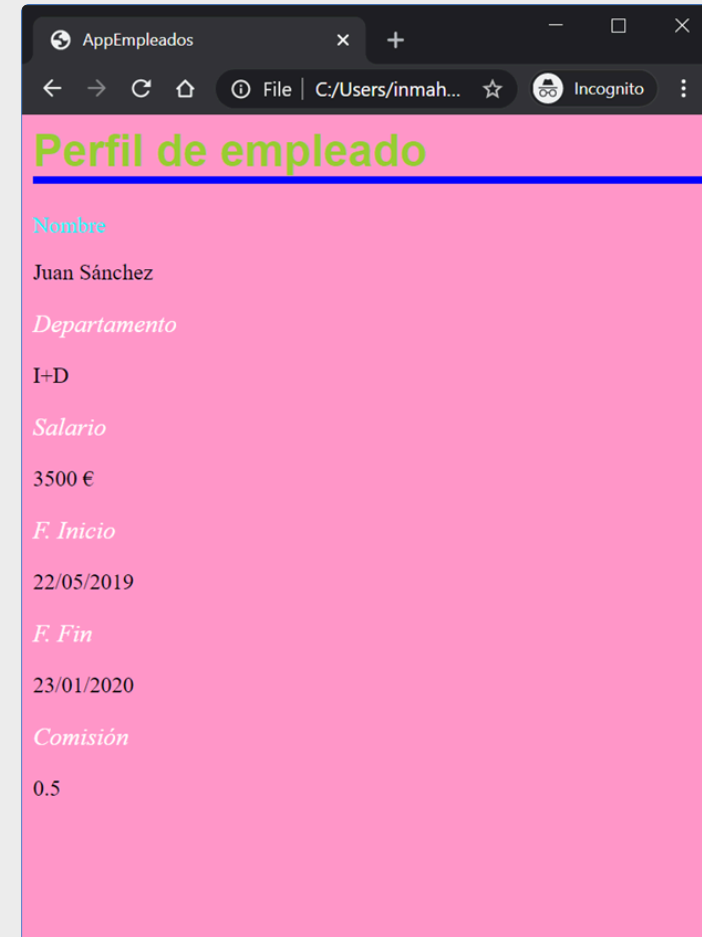


Ejemplo: selectores **#id**

```
<html>
<head>
  ...
</head>
<body>
<h1>Perfil de empleado</h1>
<p id="nombre">Nombre</p> Juan Sánchez
<p class="rotulo"> Departamento</p> I+D
  ...
</body>
</html>
```

```
...
/* empleado.css */
#nombre {
  color: cyan;
  font-style: bold;
}
```

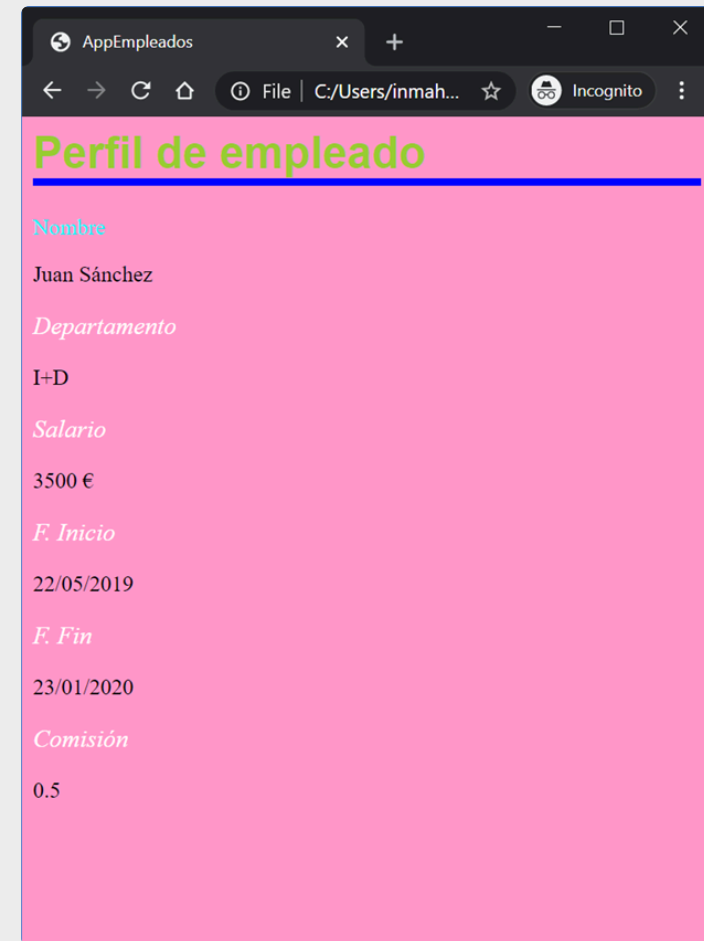
Los ids deberin ser únicos, si no lo son se aplica el estilo a **TODOS** los elementos con ese id



Ejemplo: selector universal *

```
<html>
  <head>
    <title>AppEmpleados</title>
    <link rel="stylesheet" type="text/css"
href="empleado.css" />
  </head>
  <body>
    <h1 class="rotulo">Perfil de empleado</h1>
    <p id="nombre">Nombre</p> <p>Juan Sánchez</p>
    <p id="nombre"> Departamento</p> <p>I+D</p>
    <p class="rotulo">Salario</p> <p>3500 €</p>
    <p class="rotulo">F. Inicio</p> <p>22/05/2019</p>
    <p class="rotulo">F. Fin</p> <p>23/01/2020</p>
    <p class="rotulo">Comisión</p> <p>0.5</p>
  </body>
</html>
```

```
...
/* empleado.css */
* {
  background-color: #ff99cc
}
...
```



Pseudo clases

Las pseudoclasas de hiperenlace permiten especificar estilos para los distintos estados de los enlaces.

- `a:link` para los enlaces no visitados
- `a:active` para los enlaces que se están procesando
- `a:visited` para los enlaces visitados

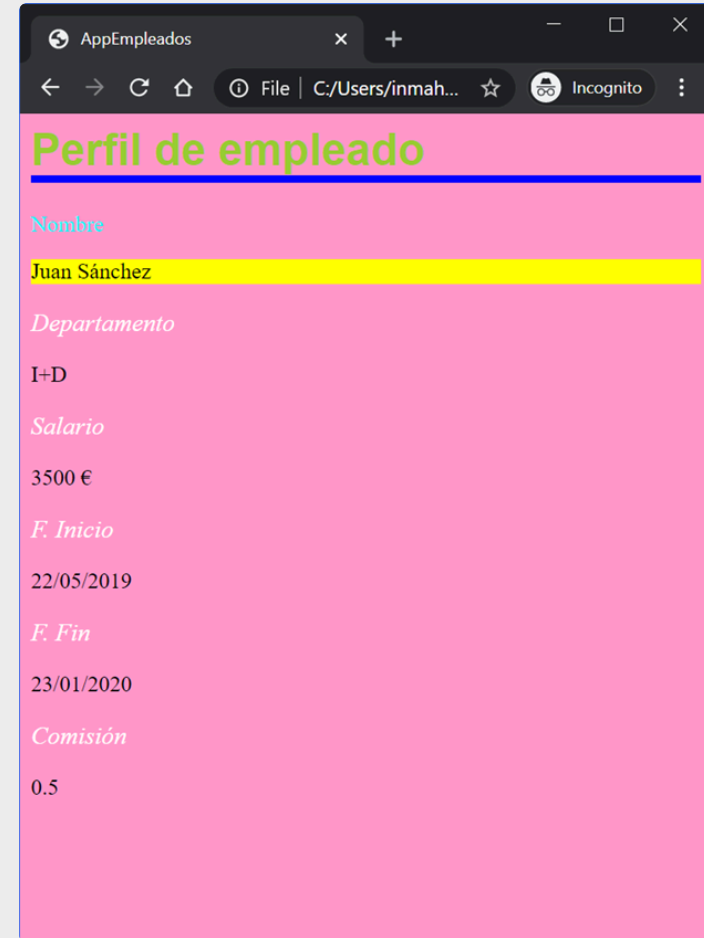
Otras pseudoclasas son aplicables a cualquier elemento

- `:hover` cuando pasa el cursor del ratón por encima
- `:focus` cuando tiene el foco (entrada del teclado)

Ejemplo: selector :hover

```
<html>
  <head>
    <title>AppEmpleados</title>
    <link rel="stylesheet" type="text/css"
      href="empleado.css" />
  </head>
  <body>
    <h1 class="rotulo">Perfil de empleado</h1>
    <p id="nombre">Nombre</p> <p>Juan Sánchez</p>
    <p id="nombre"> Departamento</p> <p>I+D</p>
    <p class="rotulo">Salario</p> <p>3500 €</p>
    <p class="rotulo">F. Inicio</p> <p>22/05/2019</p>
    <p class="rotulo">F. Fin</p> <p>23/01/2020</p>
    <p class="rotulo">Comisión</p> <p>0.5</p>
  </body>
</html>
```

```
/* empleado.css */
...
p:hover {
  background-color: yellow;
}
...
```



Cascada

Un mismo elemento de un documento HTML podría tener asociados varios estilos por parte de varias órdenes CSS diferentes.

Los estilos para un elemento se van acumulando conforme se va bajando en la "cascada".

Cada paso en la cascada puede redefinir un estilo definido anteriormente.

El esquema de prioridad es predecible (determinista).

ORDEN

1. Modificador !important
2. Atributo "style"
3. Selectores de ID (#xxx)
4. Selectores de clases (.xxx)
5. Selectores de etiquetas
6. Estilos heredados del elemento padre

Valores de propiedades

Palabras clave para propiedades no numéricas:
`underline` , `small` , `xx-large` , `right` , ...

Longitudes: pueden tener las siguientes unidades

- `em` : anchura de una "m" en el tipo de letra actual
- `ex` : altura de una "x" en el tipo de letra actual
- `px` : píxeles (puntos de pantalla)
- `in` : pulgadas (1 pulgada = 2.54 centímetros)
- `cm` , `mm` : centímetros, milímetros
- `pt` , `pc` : puntos (1/72 de pulgada), picas (12 puntos)
- `%` : porcentaje respecto al tamaño actual

URLs: URL absolutas o relativas a la hoja de estilo

- `url(protocolo://servidor:puerto/recurso)`
- `url(recurso)`

Colores: pueden expresarse como...

- Nombres estándar de color: `red` , `green` , `blue` , `aqua` , ...
- RGB hexadecimal: `#c0c0c0` , `#00ffc0` , ...
- RGB decimal: `rgb(0,255,0)` , `rgb(0%,100%,0%)`

RECOMENDACIÓN: Usar valores relativos (`%` , `em` , `ex` ...)

Propiedades del tipo de letra

`font-family` : tipo de letra; puede especificarse de forma genérica o con nombres de fuente concretos. Se puede especificar una lista por si algún tipo no está disponible.

- Genéricos: `serif`, `sans-serif`, `cursive`, `fantasy`, `monospace`
- Concretos: `Times New Roman`, `Palatino`, `Courier`, ...

`font-size` : tamaño de la letra; puede especificarse de forma absoluta mediante valores o símbolos, o de forma relativa.

- Absoluta (valores): `8pt`, `10pt`, `12pt`, `14pt`, `16pt`, ...

- Absoluta (símbolos): `xx-small`, `x-small`, `small`, `medium`, `large`, `x-large`, `xx-large` (son los valores de tamaño del navegador).
- Relativa: `120%` (un 20% más grande que la fuente del padre), `larger` (un nivel mayor), `smaller` (un nivel menor).

`font-style` : inclinación de la letra; puede ser `normal`, `italic`, `oblique`.

`font-weight` : grosor de la letra; puede ser `normal`, `bold`, `bolder`, `lighter` o un valor múltiplo de `100` entre `100` y `900` (`400` es el valor normal).

RECOMENDACIÓN: Usar valores relativos o simbólicos (`serif`, `small`, ...)

Colores

Casi todas las etiquetas HTML pueden tener un color de primer plano y un color y/o una imagen de fondo.

A diferencia de otras propiedades, el fondo no se hereda.

Especificación de colores

- Como ya se vio, los colores pueden especificarse mediante nombres de colores, en hexadecimal o en decimal (0..255 o porcentaje entre 0% y 100%).
- Selector de colores:
https://www.w3schools.com/colors/colors_picker.asp

Los 16 nombres de colores estándar son los siguientes:

	black = #000000		green = #008000
	silver = #c0c0c0		lime = #00ff00
	gray = #808080		olive = #808000
	white = #ffffff		yellow = #ffff00
	maroon = #800000		navy = #000080
	red = #ff0000		blue = #0000ff
	purple = #800080		teal = #008080
	fuchsia = #ff00ff		aqua = #00ffff

Propiedades del fondo

`background-color` : color del fondo del elemento; por defecto es `transparent` .

`background-image` : imagen para el fondo del elemento especificada como una URL; por defecto es `none` .

`background-attachment` : indica si la imagen de fondo se debe desplazar con el elemento (`scroll` , valor por defecto) o debe permanecer fija en la ventana del navegador (`fixed`).

`background-position` : posición de la imagen de fondo dentro del área del elemento; por defecto es la esquina superior izquierda. Se puede especificar un desplazamiento en píxeles, en porcentaje del área del elemento o con los valores `left` , `center` , `right` , `top` , `bottom` . Por ejemplo

- `background-position: 5px 5px; /* offset horizontal vertical */`
- `background-position: right center;`

`background-repeat` : indica como debe repetirse la imagen para completar toda el área del elemento.

- `repeat` : valor por defecto, genera un mosaico.
- `repeat-x` , `repeat-y` : repite solo en uno de los ejes.
- `no-repeat` : coloca la imagen sólo una vez.

Propiedades del texto

`text-align` : alineación del texto de un elemento; los valores válidos son `left` (por defecto), `right`, `center` y `justify`.

`text-indent` : especifica la sangría de la primera línea del texto de un elemento.

`text-decoration` : añade "decoraciones" al texto como `underline`, `overline`, `line-through`, `blink` o `none` (por defecto).

`text-transform` : permite cambiar a mayúsculas o minúsculas un texto; los valores válidos son: `capitalize` (primera letra en mayúsculas), `uppercase`, `lowercase` o `none`.

`vertical-align` : posición vertical del texto dentro del área del elemento; los valores válidos son: `baseline`, `middle`, `sub`, `super`, `text-top`, `text-bottom`, `top`, `bottom` y porcentajes.

`line-height` : separación entre las líneas del texto; son válidos valores absolutos, porcentajes o el valor normal.

`word-spacing` : espacio entre palabras; el valor por defecto es normal.

`letter-spacing` : espacio entre las letras; el valor por defecto es normal.

Propiedades de listas

`list-style-image` : especifica la URL de la imagen del marcador de los elementos o `none` si no se desea ninguno.

`list-style-position` : indica si el marcador de cada elemento debe ir dentro (`inside`) o fuera (`outside`) de la caja del elemento.

`list-style-type` : especifica el tipo de marcador, dependiendo del tipo de lista

- Listas no ordenadas (`<ul>`): `disc` , `circle` , `square` o `none` .
- Listas ordenadas (`<ol>`): `decimal` , `lower-roman` , `upper-roman` , `lower-alpha` , `upper-alpha` o `none` .

Otras propiedades: también se pueden especificar propiedades para crear contadores y generar un texto específico para cada elemento

Contenidos

Introducción

Hojas de estilo en cascada (CSS)

Sintaxis CSS

Maquetación CSS



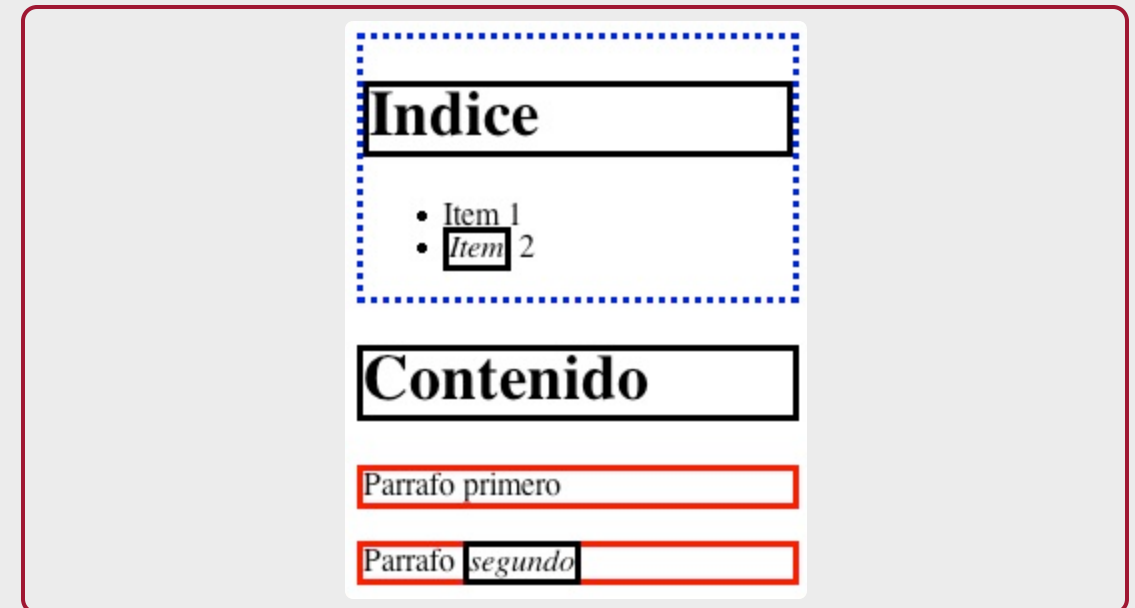
Maquetación CSS (CSS layout)

Normal flow: Cajas concéntricas.

Dos tipos

- En línea / horizontal
- De bloque / vertical

```
<body>
  <div id="menu">
    <h1>Indice</h1>
    <ul>
      <li class="aviso">Item 1</li>
      <li><em>Item</em> 2</li>
    </ul>
  </div>
  <div id="contenido">
    <h1>Contenido</h1>
    <p>Parrafo primero</p>
    <p class="aviso">Parrafo <em>segundo</em></p>
  </div>
</body>
```



https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout

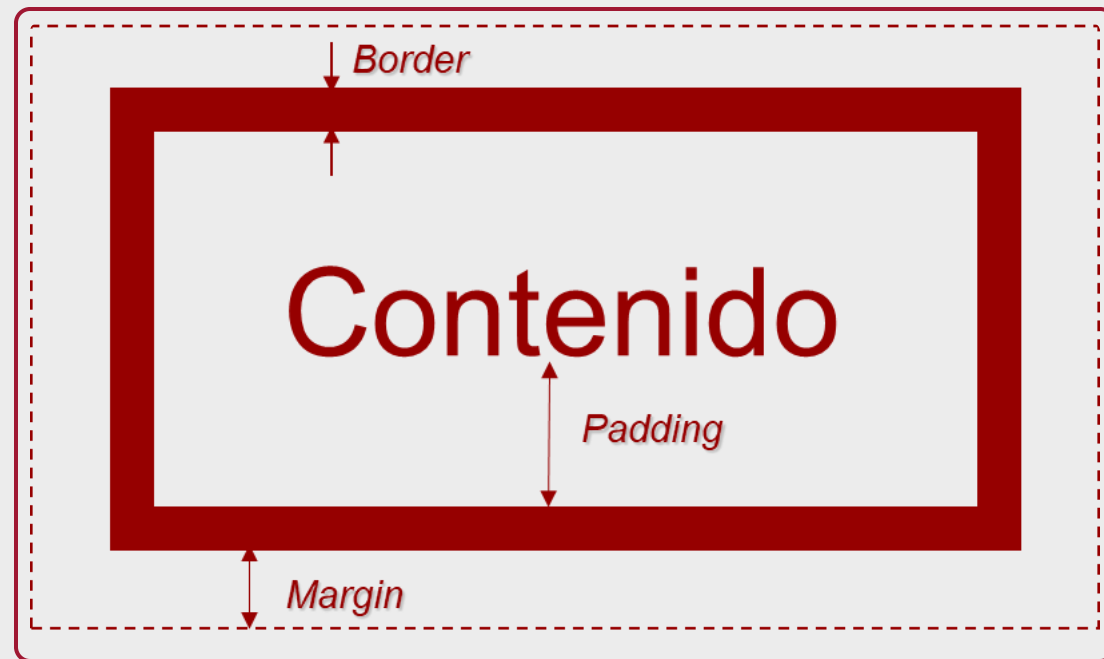
Caja de un elemento

Todos los elementos de las páginas se representan mediante cajas rectangulares.

Cada vez que se inserta una etiqueta HTML, se crea una nueva caja rectangular que encierra los contenidos de ese elemento

Padding: Espacio entre contenido y borde

Margin: Espacio entre borde y siguiente caja



Estilos de caja

Propiedades del área de contenido

- `height` : altura del área de contenido de un elemento; por defecto es `auto` , que es la altura necesaria para mostrarlo.
- `width` : igual que `height` , pero aplicado a la anchura.

Propiedades de relleno/márgenes de cajas*

- `padding-{tlbr}` : anchura del relleno o `auto` , que normalmente es el valor por defecto, que es `0` . También se puede especificar un porcentaje (con respecto a la anchura del área de contenido). El relleno se visualiza con el fondo del elemento.
- `margin-{tlbr}` : igual que `padding-{tlbr}` , pero referido a los márgenes. Los márgenes son transparentes, permitiendo ver el fondo del elemento contenedor.

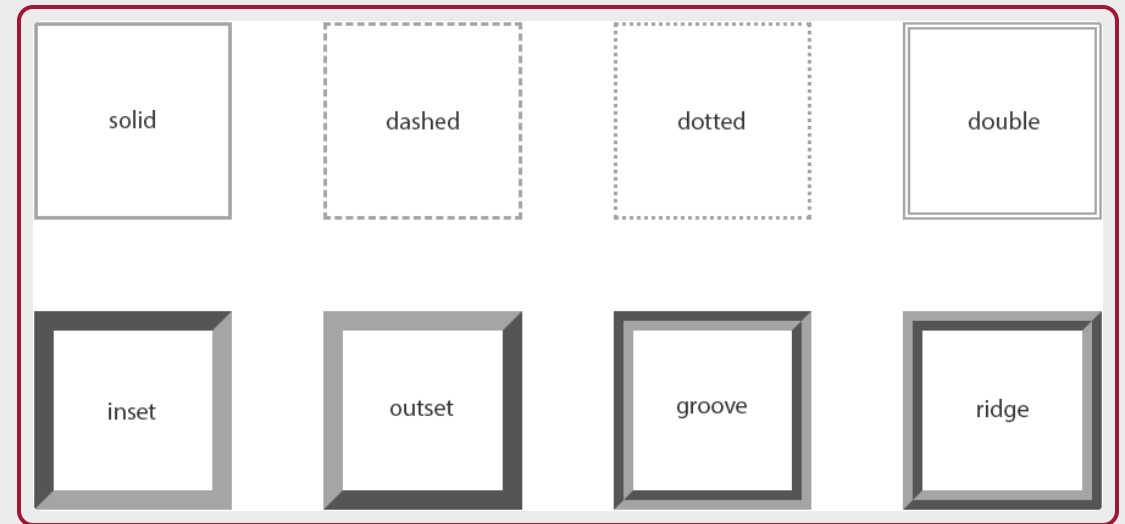
`{tlbr}` puede ser cualquiera de `top` , `left` , `bottom` , `right` .

Propiedades de bordes de cajas

`border-{tlbr}-color` : color del borde; si no se especifica, se usa el color del elemento.

`border-{tlbr}-width` : anchura del borde en valores concretos o `thin`, `medium` (por defecto), `thick`.

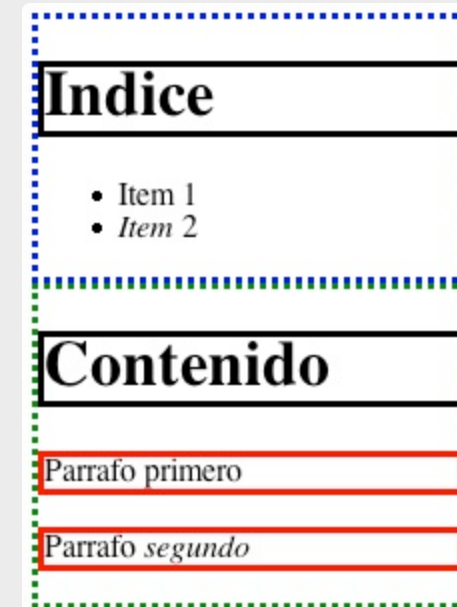
`border-{tlbr}-style` : indica el estilo del borde, que puede ser uno de los siguientes: `none`, `solid`, `dashed`, `dotted`, `double`, `inset`, `outset`, `groove`, `ridge`.



Normal flow

Unas cajas debajo de otras

```
<html>
  <head>...</head>
  <body>
    <div id="menu">
      <h1>Indice</h1>
      <ul>
        <li>Item 1</li>
        <li><em>Item</em> 2</li>
      </ul>
    </div>
    <div id="contenido">
      <h1>Contenido</h1>
      <p>Parrafo primero</p>
      <p>Parrafo <em>segundo</em></p>
    </div>
  </body>
</html>
```



Alteración del normal flow

Las propiedades `display` y `visibility` : mostrar/ocultar ciertas cajas

Flotación: Hacer que un bloque de elementos se agrupen a uno u otro lado

Propiedad `position`: para fijar la posición relativa de una caja dentro de otra caja



Propiedades display y visibility

`display` : indica cómo visualizar un elemento;

- Sus valores más habituales son `block` , `inline` , `list-item` , `flex` , `grid` y `none` .
- Listado completo de valores: https://www.w3schools.com/cssref/pr_class_display.asp
- Si se especifica `none` , no se muestra ni se tiene en cuenta para la maquetación al elemento ni a sus hijos.
- Cada elemento tiene un valor por defecto de `display` ; por ejemplo, los `<li>` tienen valor `block` , mientras que los `<a>` tienen valor `inline` . Se puede cambiar el comportamiento de estos elementos cambiando el valor de `display` .
- Algunos valores no son compatibles con todos los navegadores

`visibility` : determina la visibilidad de un elemento.

- Sus valores son `visible` , `hidden` , `initial` , `inherit` .
- El valor `hidden` hace invisible al elemento, pero sigue teniéndolo en cuenta para la maquetación de la página (deja el hueco).

Flotación

Flotar un elemento cambia el comportamiento de ese elemento y de los bloques que le siguen en normal flow

El elemento se mueve a izquierda o derecha, y los elementos que lo rodean flotan a su alrededor

- Similar al texto e imágenes en un periódico.

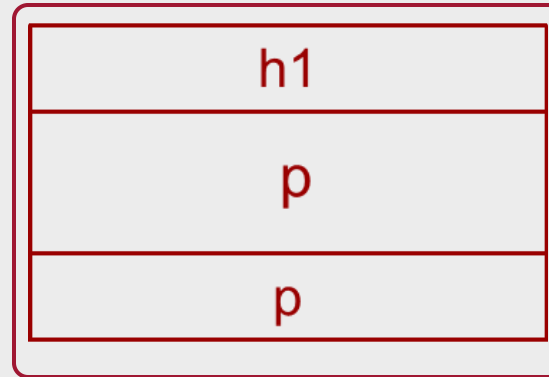
Eso quiere decir que

- Deja de ocupar todo el ancho del documento
- Se sale del orden normal y los de abajo ocupan su lugar

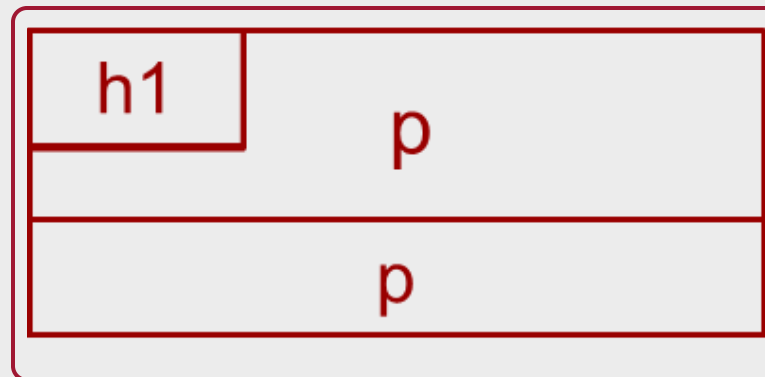
Los valores para la propiedad float son: `none`, `left`, `right`, `initial`, `inherit`

En la actualidad, es menos común usar floats para organizar elementos en filas y columnas que `flexbox` o `grid`

Flotación - ejemplo



```
h1{  
  float: left;  
}
```



Ejemplo de flotación

Indice

- Item 1
- *Item 2*

Contenido

Parrafo primero

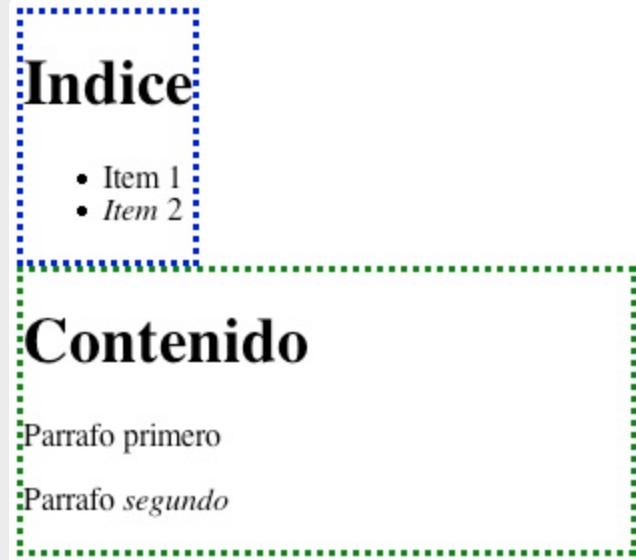
Parrafo *segundo*

```
#menu {  
  float: left;  
}
```

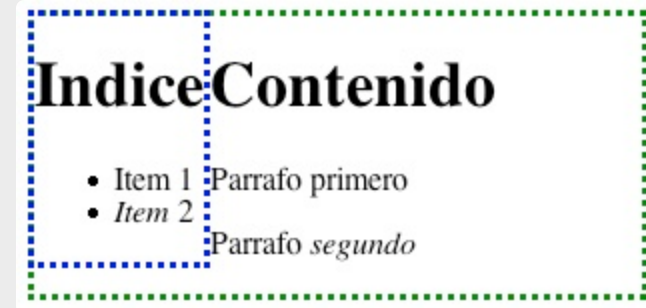
IndiceContenido

- Item 1 Parrafo primero
- *Item 2* Parrafo *segundo*

Flotación



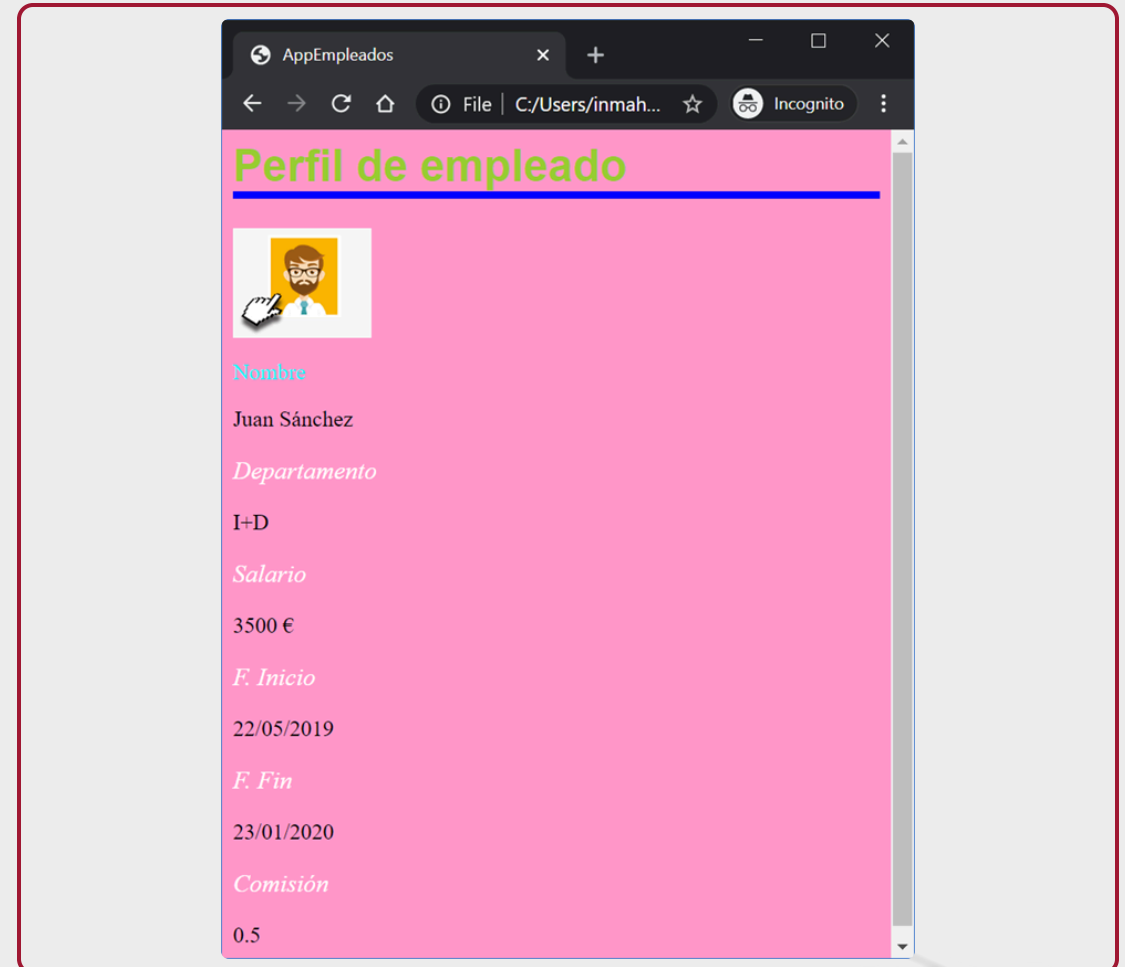
```
#contenido {  
  clear: left;  
}
```



Flotar un elemento afecta también a los elementos que le siguen. `clear` lo evita.

Flotación: ejemplo (I)

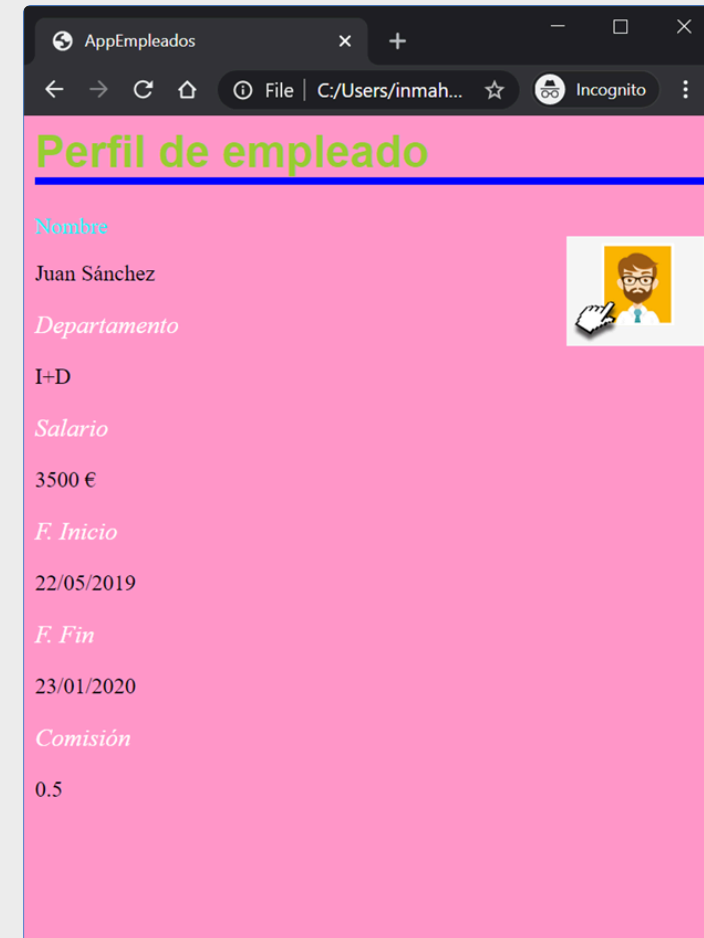
```
(...)  
<h1 class="rotulo">Perfil de empleado</h1>  
  
<p id="nombre">Nombre</p><p>Juan Sánchez</p>  
<p class="rotulo">Departamento</p><p>I+D</p>  
(...)
```



Flotación: ejemplo (II)

```
(...)  
<h1 class="rotulo">Perfil de empleado</h1>  
  
<p id="nombre">Nombre</p><p>Juan Sánchez</p>  
<p class="rotulo">Departamento</p><p>I+D</p>  
(...)
```

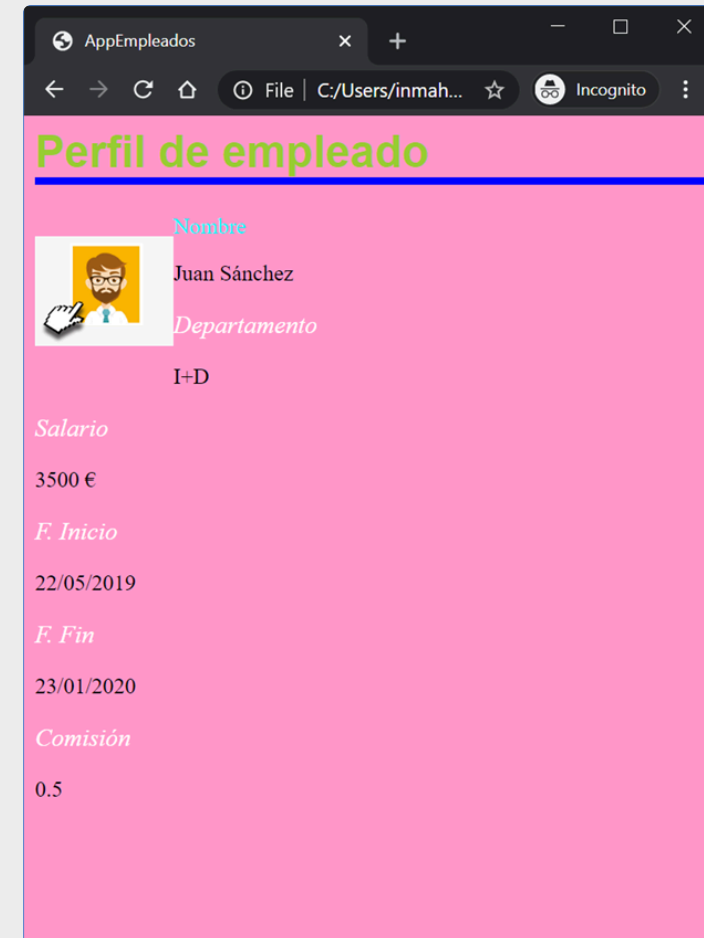
```
.foto {  
  float: right;  
}
```



Flotación: ejemplo (III)

```
(...)  
<h1 class="rotulo">Perfil de empleado</h1>  
  
<p id="nombre">Nombre</p><p>Juan Sánchez</p>  
<p class="rotulo">Departamento</p><p>I+D</p>  
(...)
```

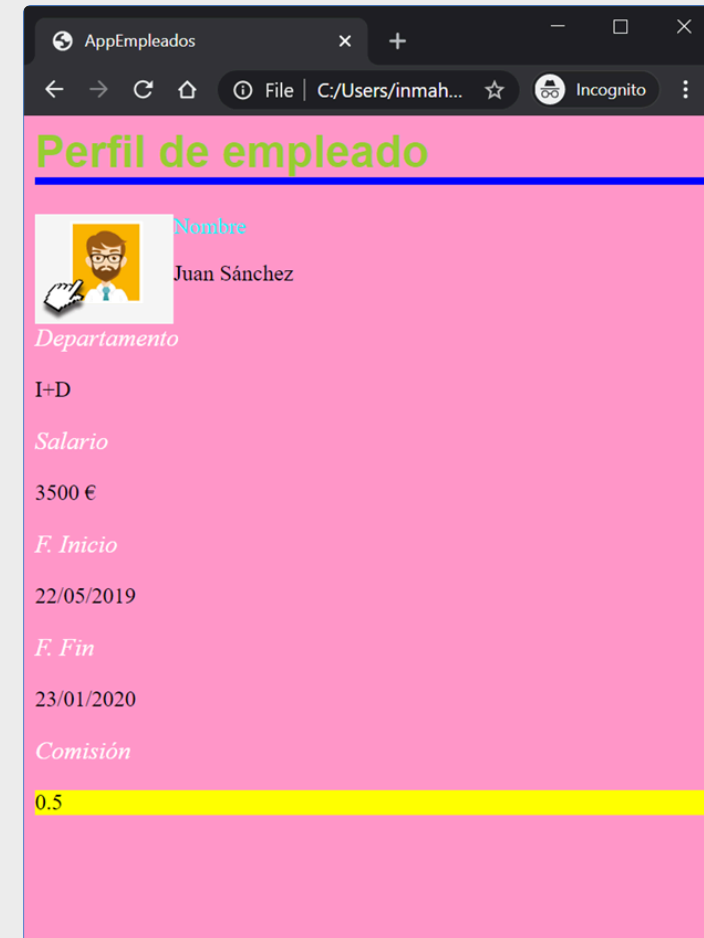
```
.foto {  
  float: left;  
}
```



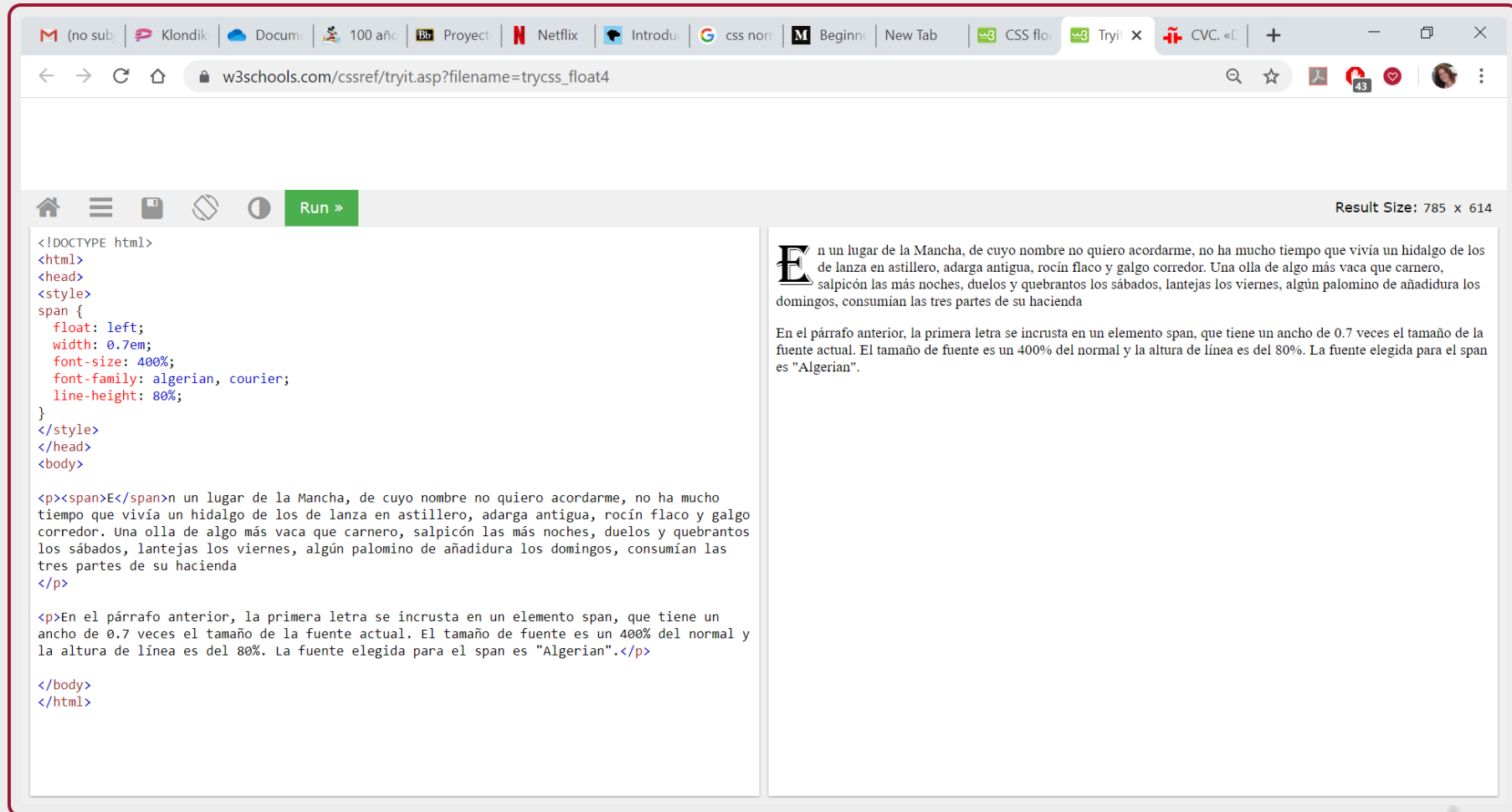
Flotación: ejemplo (IV)

```
(...)  
<h1 class="rotulo">Perfil de empleado</h1>  
  
<p id="nombre">Nombre</p><p>Juan Sánchez</p>  
<p class="rotulo">Departamento</p><p>I+D</p>  
(...)
```

```
p.rotulo {  
  color: white;  
  font-style: italic;  
  font-size: large;  
  clear: left;  
}  
#nombre {  
  color: cyan;  
  font-style: bold;  
}  
.foto {  
  float: right;  
}
```



Flotación – otro ejemplo



Propiedad position

Permite indicar la posición exacta en la que debe colocarse un elemento

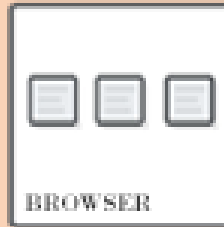
Tipos de posicionamiento

- **Estático:** por defecto, el elemento se coloca en la posición que le corresponde según el normal flow.
- **Relativo:** Permite modificar la posición de un elemento de forma relativa a su posición en el normal flow, incluso solapándose con otros elementos.
- **Absoluto:** Permite mover un elemento completamente fuera del normal flow, por ejemplo a una posición relativa a los bordes del documento html
- **Fijo:** Es muy parecido al absolute, pero fija un elemento con respect al viewport del navegador, no con respecto a otro elemento.
- **Sticky:** el elemento usa posicionamiento estático hasta que llega a una determinada posición del viewport, momento en el que actúa con posicionamiento fijo

En resumen

A QUICKLOOK OF CSS POSITION

STATIC



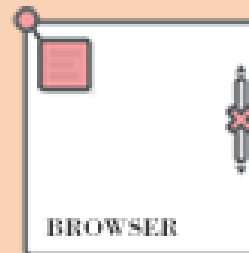
RELATIVE



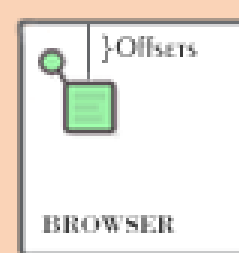
ABSOLUTE



FIXED



STICKY

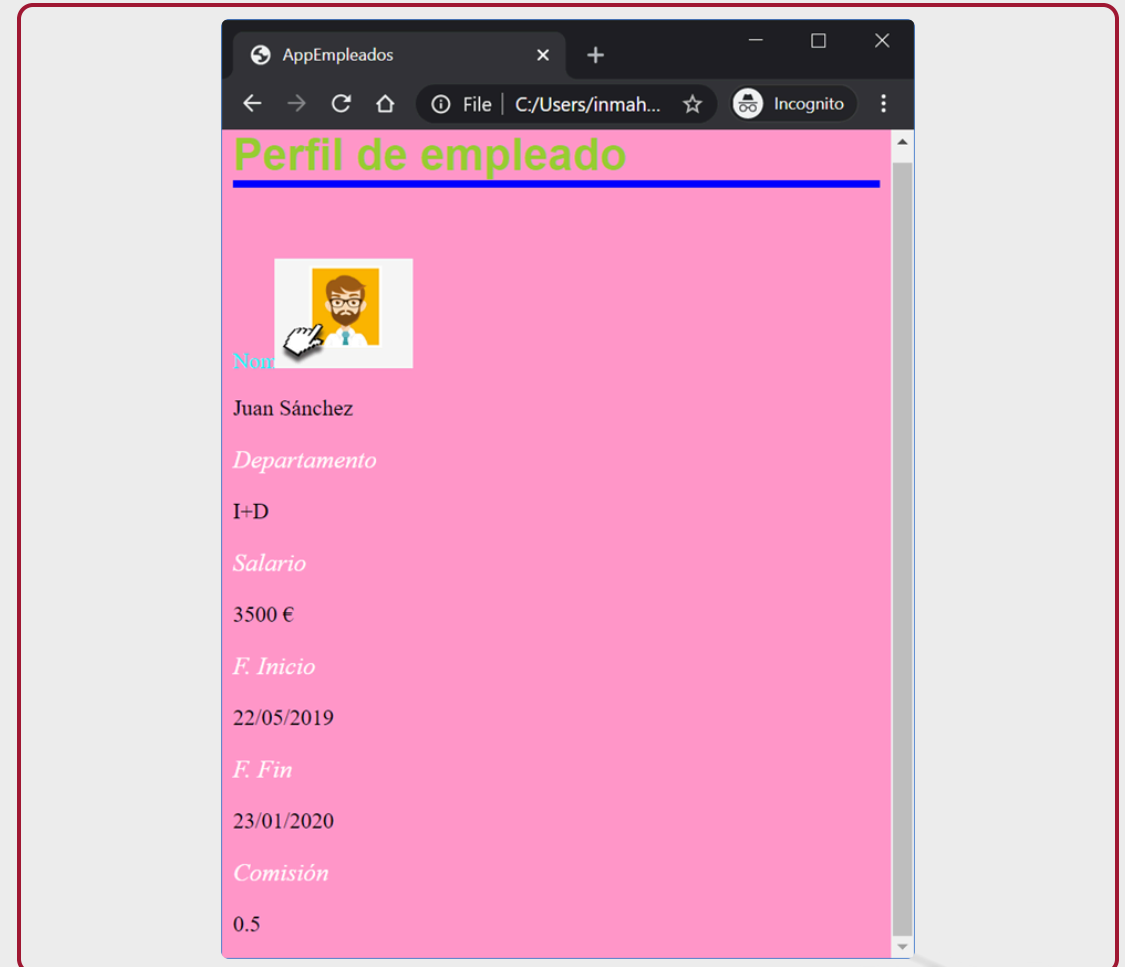


cleo{N}XX
PERFORMANCE ++

Position: ejemplo (I)

```
(...)  
<h1 class="rotulo">Perfil de empleado</h1>  
  
<p id="nombre">Nombre</p><p>Juan Sánchez</p>  
<p class="rotulo">Departamento</p><p>I+D</p>  
(...)
```

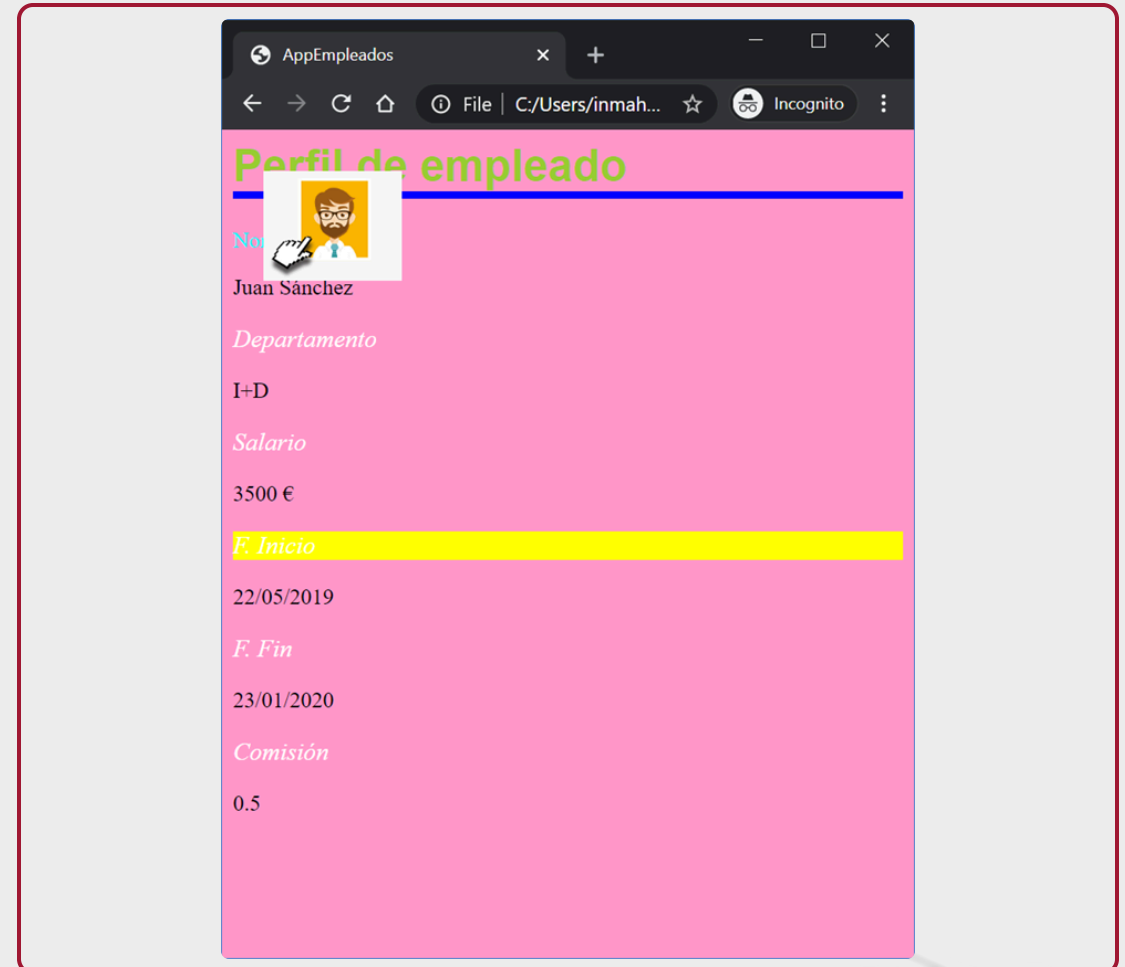
```
.foto {  
  position: relative;  
  top: 30px;  
  left: 30px;  
}
```



Position: ejemplo (II)

```
(...)  
<h1 class="rotulo">Perfil de empleado</h1>  
  
<p id="nombre">Nombre</p><p>Juan Sánchez</p>  
<p class="rotulo">Departamento</p><p>I+D</p>  
(...)
```

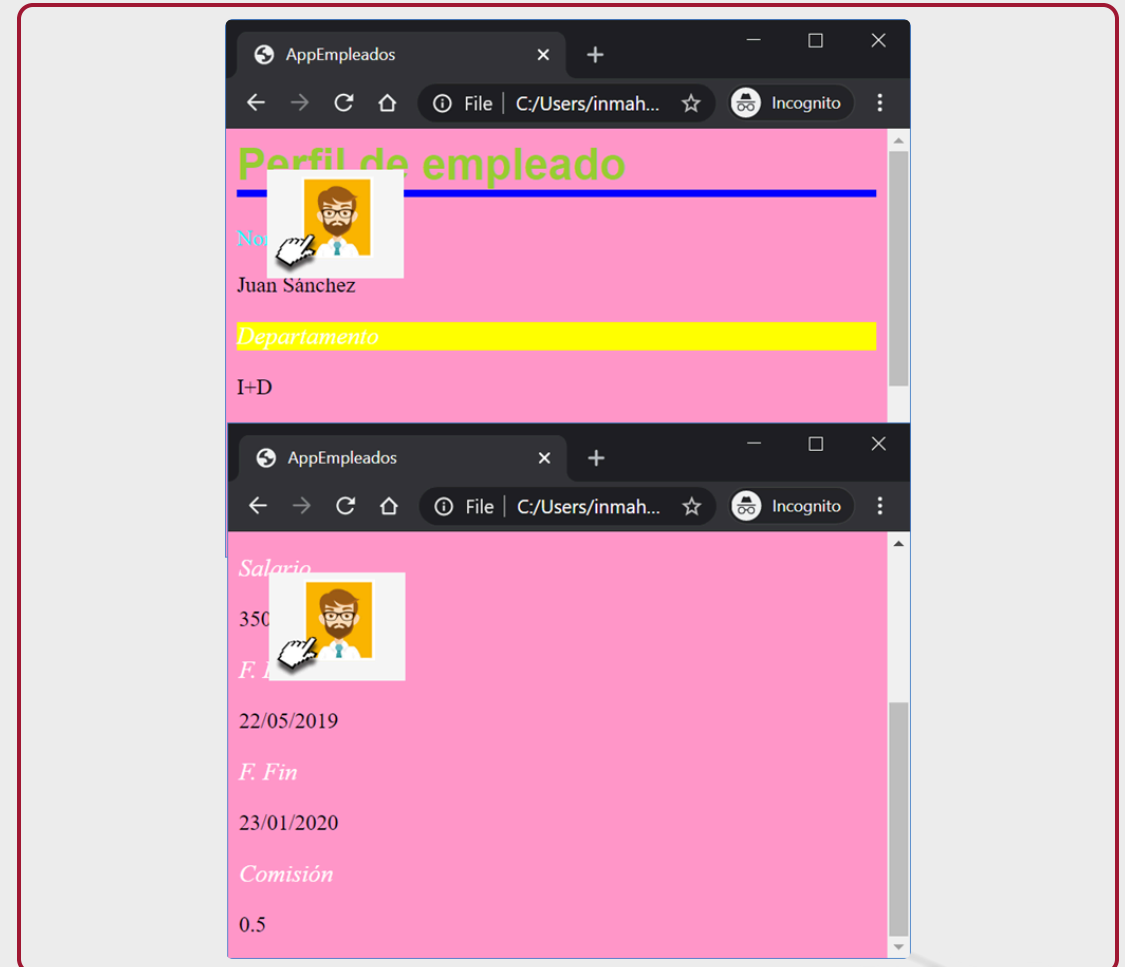
```
.foto {  
  position: absolute;  
  top: 30px;  
  left: 30px;  
}
```



Position: ejemplo (III)

```
(...)  
<h1 class="rotulo">Perfil de empleado</h1>  
  
<p id="nombre">Nombre</p><p>Juan Sánchez</p>  
<p class="rotulo">Departamento</p><p>I+D</p>  
(...)
```

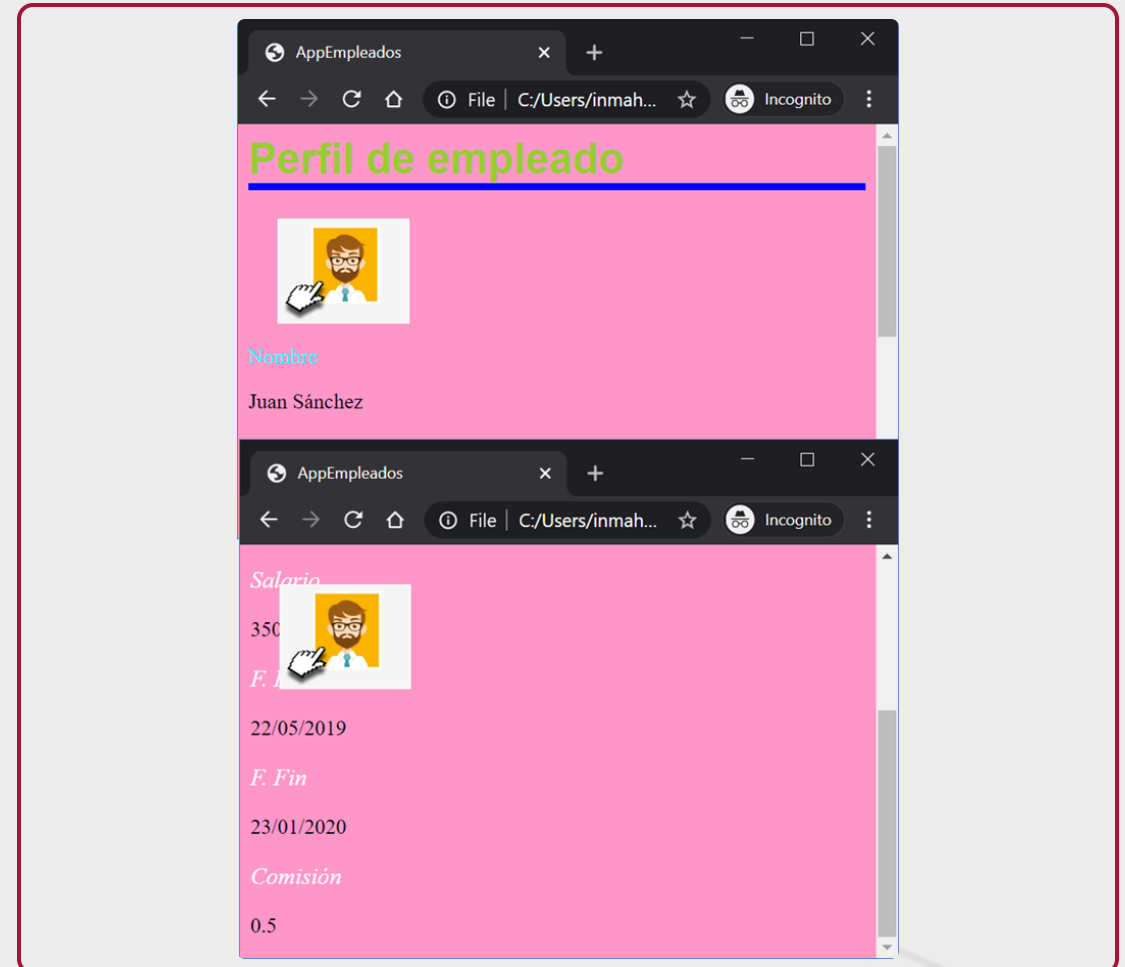
```
.foto {  
  position: fixed;  
  top: 30px;  
  left: 30px;  
}
```



Position: ejemplo (IV)

```
(...)  
<h1 class="rotulo">Perfil de empleado</h1>  
  
<p id="nombre">Nombre</p><p>Juan Sánchez</p>  
<p class="rotulo">Departamento</p><p>I+D</p>  
(...)
```

```
.foto {  
  position: sticky;  
  top: 30px;  
  left: 30px;  
}
```

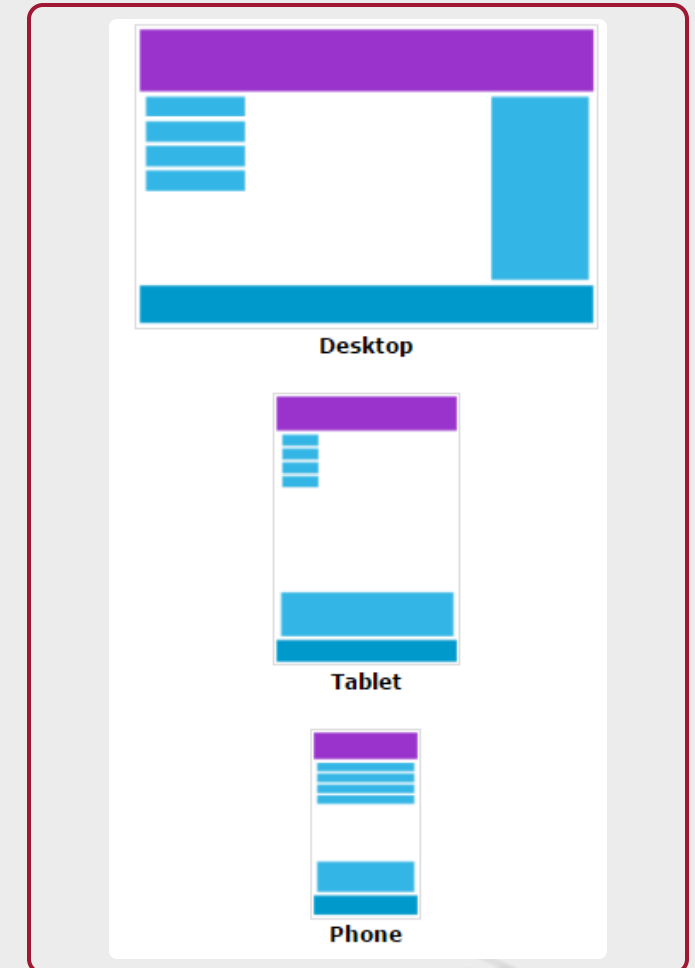


CSS3 & Responsive Web Design (RWD)

CSS3 no es soportado del mismo modo por todos los navegadores (Inestabilidad de propiedades)

Limitaciones del normal flow: Al crear layouts complejos con elementos colocados en distintas filas y columnas

Responsive Web Design (RWD): Recomendaciones de diseño para conseguir que la misma página se visualice de forma diferente para adaptarse de forma óptima al tamaño de pantalla de cada dispositivo.



Responsive Web Design (RWD)

Se lleva a cabo sólo con HTML y CSS (CSS Responsive)

No se trata de una tecnología concreta, sino de un conjunto de buenas prácticas a la hora de diseñar el layout de las páginas web para que éstas puedan responder al dispositivo que se usa para visualizarlas, es decir, ser capaces de adaptarse a él.

Conceptos a manejar en **RWD**

- Viewport
- Media queries
- Flexible Grids

Implementación de **RWD**

- Diseño específico de cada usuario
- Frameworks como W3.CSS o Bootstrap

Viewports

Es el área visible de una página en cada dispositivo. Se establece con

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Se recomienda:

- No utilizar elementos de ancho fijo grande
- No diseñar para un tamaño fijo de la zona de visualización
 - (Usar dimensiones relativas)
- Utilizar Media Queries para encajar estilos del mismo contenido HTML a distintos dispositivos (en función del tamaño disponible en cada uno)

Media Queries

Se pueden comprobar características del medio de presentación

- Anchura y altura del viewport
- Anchura y altura del dispositivo
- Orientación
- Resolución

y aplicar CSS condicionado a estas características

Ejemplo: si se está visualizando la página en una pantalla y el viewport es de al menos 480 píxeles, mostrar el fondo de la página en color verde claro

```
@media screen and (min-width: 480px) {  
  body {  
    background-color: lightgreen;  
  }  
}
```

Media Queries

Se establecen break-points (reglas válidas en esa situación) para cada situación.

- Se empieza diseñando para el dispositivo de pantalla más pequeña
- Un break-point aplicable sobre el grid, con clases específicas por categoría de dispositivo
- Lo más común: crear un layout de una sola columna para dispositivos estrechos (móviles), y después ir comprobando con media queries el tamaño para ir adaptando el tamaño de las múltiples columnas

```
/* For mobile phones: */
[class*="col-"] {
  width: 100%;
}
@media only screen and (min-width: 600px) {
  /* For tablets: */
  .col-m-1 {width: 8.33%;}
  .col-m-2 {width: 16.66%;}
  .col-m-3 {width: 25%;}
  .col-m-4 {width: 33.33%;}
  .col-m-5 {width: 41.66%;}
  .col-m-6 {width: 50%;}
  .col-m-7 {width: 58.33%;}
  .col-m-8 {width: 66.66%;}
  .col-m-9 {width: 75%;}
  .col-m-10 {width: 83.33%;}
  .col-m-11 {width: 91.66%;}
  .col-m-12 {width: 100%;}
}
@media only screen and (min-width: 768px) {
  /* For desktop: */
  .col-1 {width: 8.33%;}
  .col-2 {width: 16.66%;}
  .col-3 {width: 25%;}
  .col-4 {width: 33.33%;}
  .col-5 {width: 41.66%;}
  .col-6 {width: 50%;}
  .col-7 {width: 58.33%;}
  .col-8 {width: 66.66%;}
  .col-9 {width: 75%;}
  .col-10 {width: 83.33%;}
  .col-11 {width: 91.66%;}
  .col-12 {width: 100%;}
}
```

Flexible Grids

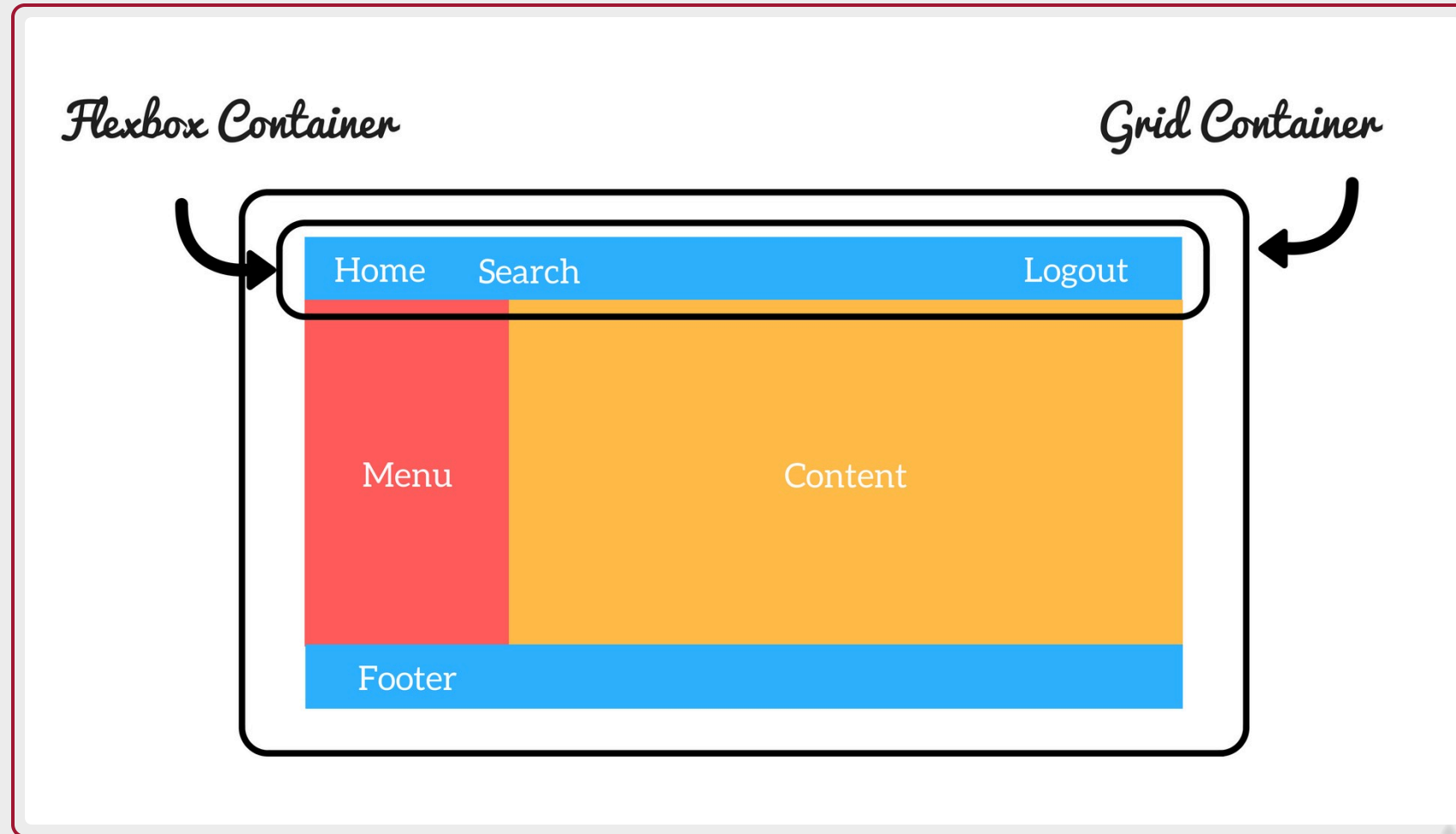
Las páginas RWD no solo tienen usan media queries, también se basan en los grids flexibles.

Un grid flexible significa que no es necesario definir breakpoints para todos los posibles tamaños de dispositivos que existen (sería imposible). Se crea un breakpoint y se cambia el diseño en el momento en que el contenido deja de verse bien.

Implementación: FlexBox (una dimension), CSS Grid (dos dimensiones)



Flexbox vs CSS Grid



Grids

```
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<style>
* {
  box-sizing: border-box;
}
.header {
  border: 1px solid red;
  padding: 15px;
}
.menu {
  width: 25%;
  float: left;
  padding: 15px;
  border: 1px solid red;
}
.main {
  width: 75%;
  float: left;
  padding: 15px;
  border: 1px solid red;
}
</style>
</head>
```

http://www.w3schools.com/css/tryit.asp?filename=tryresponsive_webpage

Chania

- The Flight
- The City
- The Island
- The Food

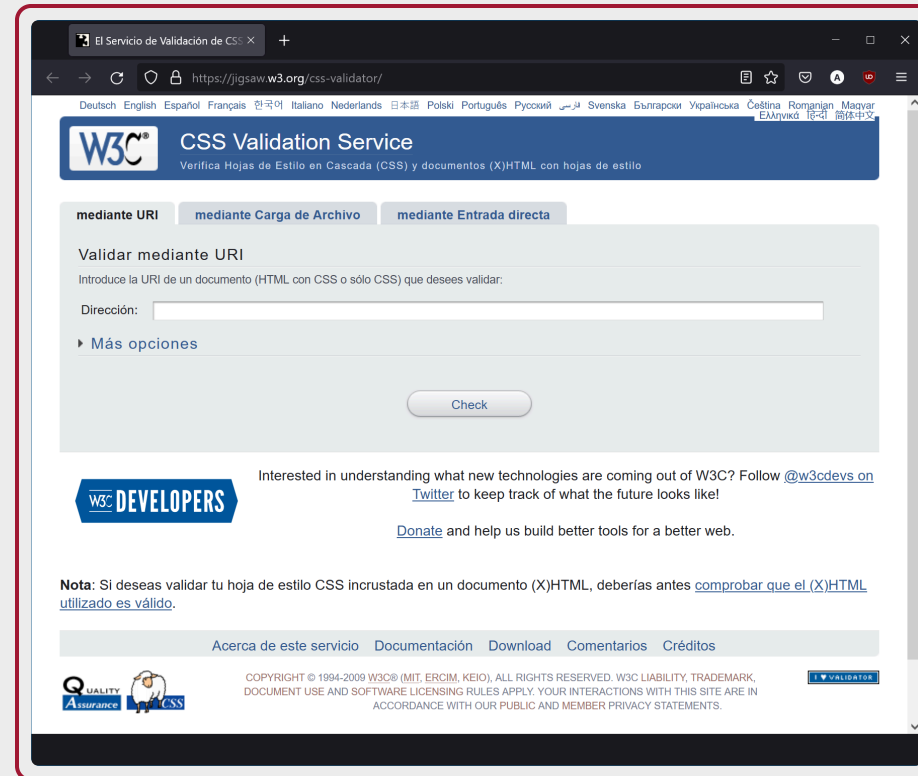
The City

Chania is the capital of the Chania region on the island of Crete. The city can be divided in two parts, the old town and the modern city.

Resize the browser window to see how the content respond to the resizing.

Validador

El W3C ofrece un servicio de validación de hojas de estilo



<http://jigsaw.w3.org/css-validator/>

Para curiosear

<https://www.awwwards.com>

<http://www.csszengarden.com>

<https://www.slideshare.net/LeaVerou/css3-a-practical-introduction-ft2010-talk>

https://tympanus.net/codrops/css_reference/

Un cuadro hecho únicamente con HTML5 y CSS



[Enlace GIT](#)

Ejercicio

Implementar la página del perfil del empleado vista en clase y experimentar con los distintos selectores/atributos/valores de propiedades



Conclusiones

CSS separa la **presentación del contenido**, lo que permite reutilizar estilos, mantener páginas más limpias y adaptar una misma información a distintos contextos.

Las reglas de estilo combinan **selectores, propiedades y valores**, y su aplicación depende de la **cascada**, la **herencia** y la **especificidad**.

Elegir bien los **selectores** y conocer los tipos de **valores** es esencial para escribir hojas de estilo correctas y predecibles.

Las propiedades de texto, color, fondo, listas y cajas permiten controlar la apariencia visual de los elementos con bastante precisión.

Conclusiones

La maquetación CSS se apoya en conceptos como **normal flow**, **float**, **display**, **position** y el **modelo de caja**.

Las técnicas modernas de diseño deben considerar la compatibilidad entre navegadores y apoyarse en principios de **Responsive Web Design (RWD)**.

El uso de herramientas de inspección y del **validador CSS** ayuda a detectar errores y a comprender mejor cómo interpreta el navegador las reglas.

Diseñar con **CSS** no consiste solo en “decorar”, sino en construir interfaces más **claras**, **adaptables** y **mantenibles**.



Gracias

Universidad de Sevilla

**Departamento de Lenguajes y Sistemas
Informáticos**

CSS is OK

UNIVERSIDAD DE SEVILLA